Big data analytic

Recommender systems

Le Thanh Van

HCMC University of Technology

Amazon: shopping online

1 item added to Cart

Data Clustering: Algorithms and Applications (Chapman & Hall/CRC Data...
by Charu C. Aggarwal
$81.17
Only 1 left in stock (more on the way).
☐ This is a gift Learn more

Order subtotal: $81.17
1 item in your Cart

Edit your Cart | Proceed to checkout

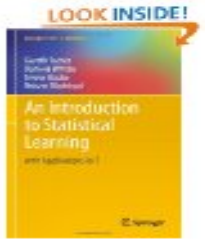
✓ Your order qualifies for FREE Shipping!
Select FREE Shipping at checkout. (Some restrictions apply)

Get the Amazon.com Rewards Visa Card and **Get $30 off instantly**

Current Total:	$ 81.17
Gift Card:	**- $ 30.00**
Cost After Savings:	**$ 51.17**

Apply now

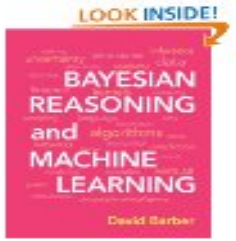
Customers Also Bought these Highly Rated Items



An Introduction to Statistical Learning: with Applications...
by Gareth James
Hardcover
★★★★★ 22
~~$79.99~~ $53.92
32 New & 14 Used from $44.92
Add to Cart

Data Science for Business
by Foster Provost
Paperback
★★★★★ 54
~~$34.99~~ $27.55
45 New & 11 Used from $26.34
Add to Cart

Python for Data Analysis
by Wes McKinney
Paperback
★★★★½ 48
~~$39.99~~ $23.99
53 New & 22 Used from $18.67
Add to Cart



Bayesian Reasoning and Machine Learning
by David Barber
Hardcover
★★★★½ 10
~~$92.00~~ $82.80
43 New & 15 Used from $65.36
Add to Cart

Amazon: shopping online

Recommended for You

These recommendations are based on items you own and more.

view: **All** | New Releases | Coming Soon

1.

An Introduction to Statistical Learning: with Applications in R (Springer Texts in Statistics)
by Gareth James (August 12, 2013)
Average Customer Review: ★★★★★ (22)
In Stock

List Price: ~~$79.99~~
Price: $53.92
46 used & new from $44.92

Add to Cart | Add to Wish List

I own it | Not interested | Rate this item

Recommended because you added **Data Clustering** to your Shopping Cart (Fix this)

2.

Data Science for Business: What you need to know about data mining and data-analytic thinking
by Foster Provost (August 19, 2013)
Average Customer Review: ★★★★★ (54)
In Stock

List Price: ~~$34.99~~
Price: $27.55
56 used & new from $26.34

Add to Cart | Add to Wish List

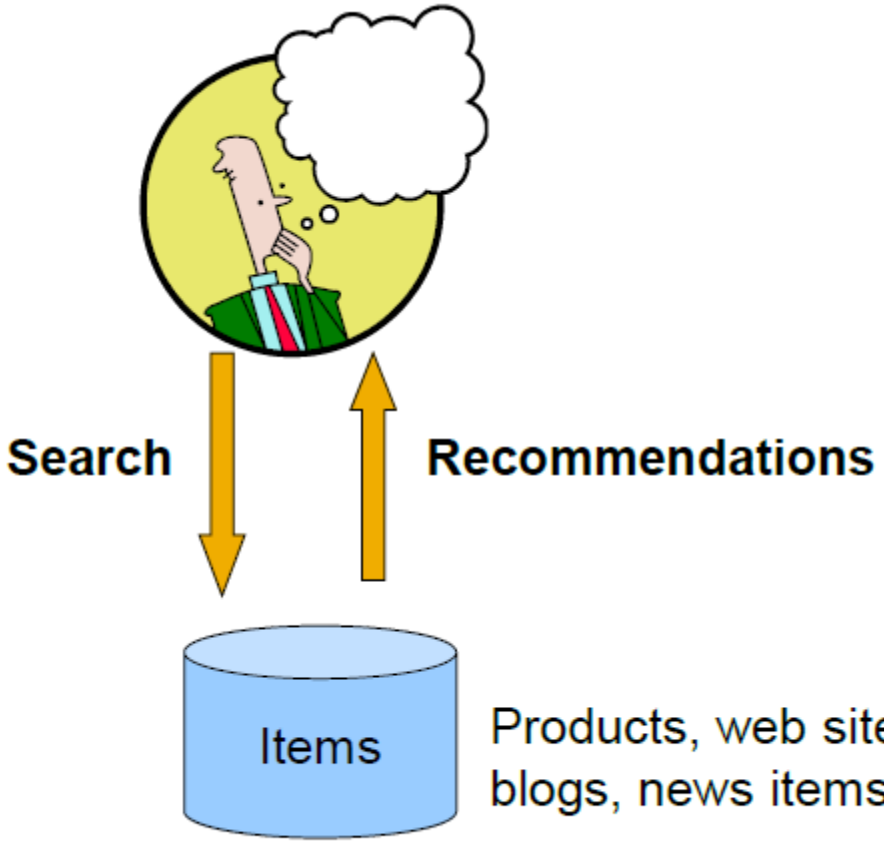
I own it | Not interested | Rate this item

Recommended because you added **Data Clustering** to your Shopping Cart (Fix this)

3.

Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython
by Wes McKinney (October 29, 2012)
Average Customer Review: ★★★★½ (48)

Recommendations



Examples:

amazon.com

PANDORA

StumbleUpon

del.icio.us

NETFLIX

movielens
helping you find the *right* movies

last·fm
the social music revolution

Google News

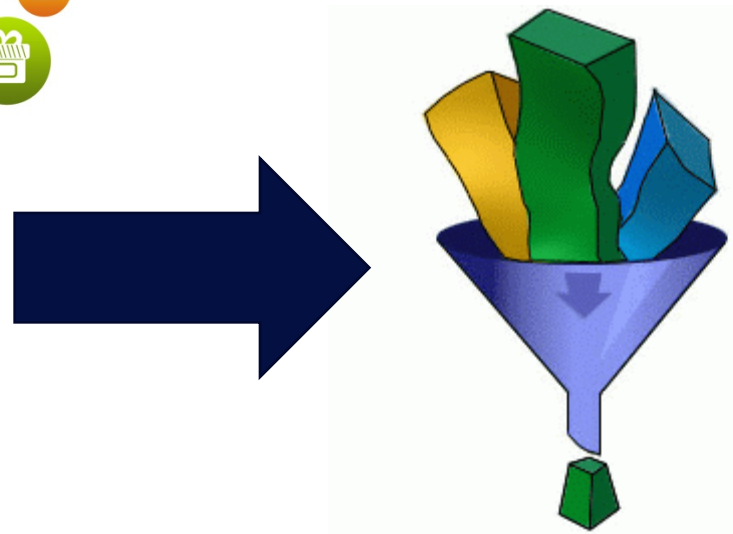
YouTube

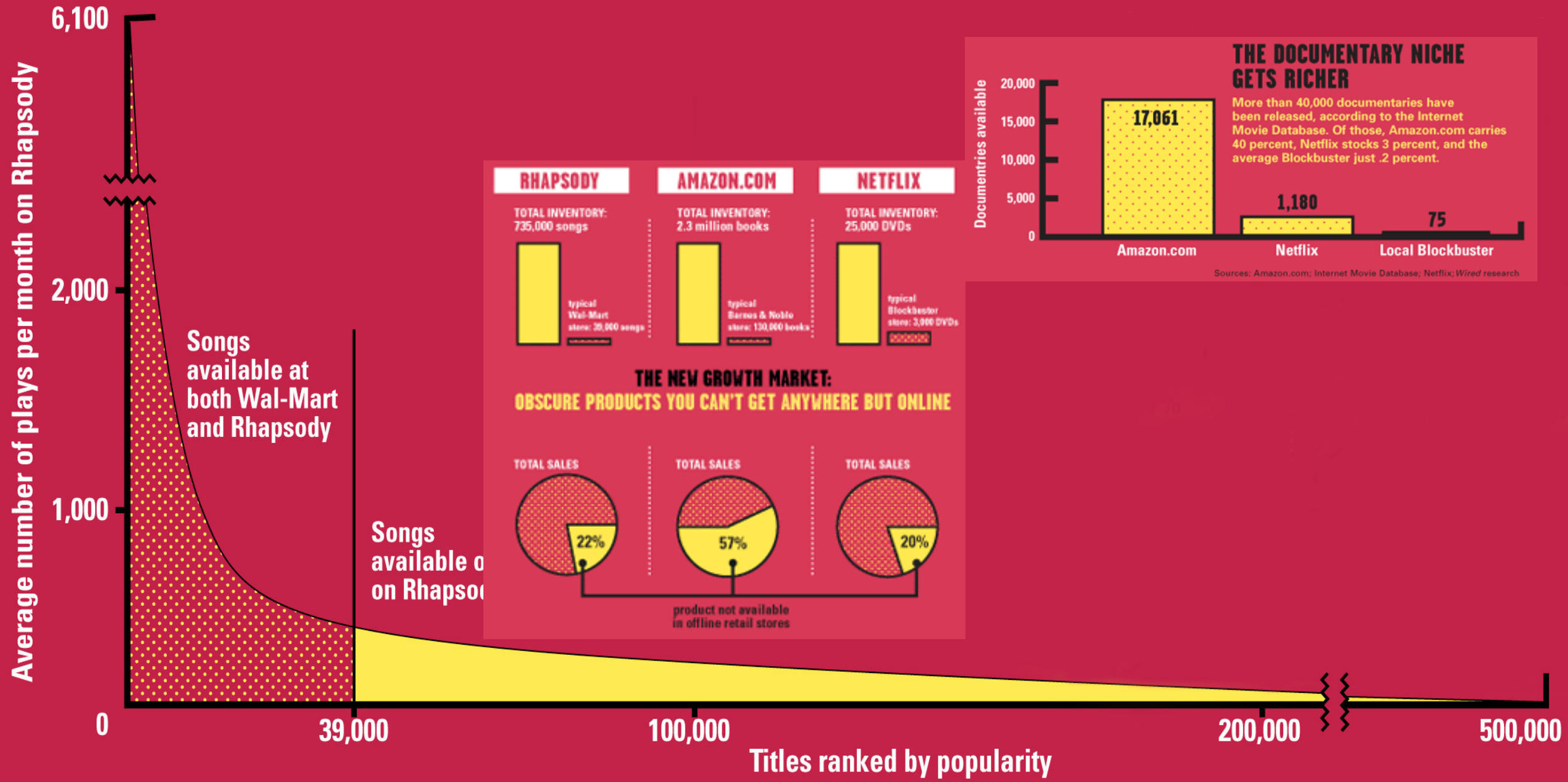
XBOX LIVE

From scarcity to Abundance

- Physical delivery system is a scarce commodity for traditional retailers
  - Also: TV networks, movie theaters,…

- Web enables near-zero-cost dissemination of information about products
  - From scarcity to abundance
- More choice necessitates better filters
  - Recommendation engines
  - How **Into Thin Air** made **Touching the Void** a bestseller: http://www.wired.com/wired/archive/12.10/tail.html





Sources: Erik Brynjolfsson and Jeffrey Hu, MIT, and Michael Smith, Carnegie Mellon; Barnes & Noble; Netflix; RealNetworks

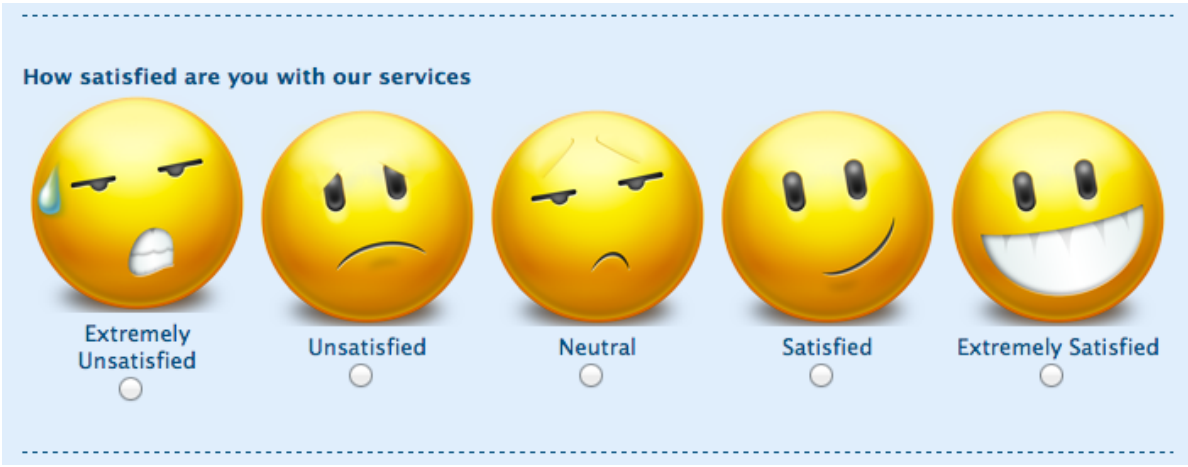
Source: Chris Anderson (2004)

Types of Recommendations

- **Editorial and hand curated**
  - List of favorites
  - Lists of “essential” items
- **Simple aggregates**
  - Top 10, Most Popular, Recent Uploads
- **Tailored to individual users Amazon, Netflix, …**

Formal model

- ***X = set of Customers***
- ***S = set of Items***
- **Utility function *u: X × S → R***
  - R = set of ratings
  - R is a totally ordered set
  - e.g., 0-5 stars, real number in [0,1]



Utility matrix

	Avatar	LOTR	Matrix	Pirates
Alice	1		0.2	
Bob		0.5		0.3
Carol	0.2		1	
David				0.4

Key problems

- Gathering “known” ratings for matrix
  - How to collect the data in the utility matrix
- Extrapolate unknown ratings from the known ones
  - Mainly interested in high unknown ratings
    - We are not interested in knowing what you don’t like but what you like
- Evaluating extrapolation methods
  - How to measure success/performance of recommendation methods

Gathering ratings

- **Explicit**
  - **Ask people to rate items**
    - Doesn't work well in practice – people can't be bothered
- **Implicit**
  - **Learn ratings from user actions**
    - E.g., purchase implies high rating
  - What about low ratings?

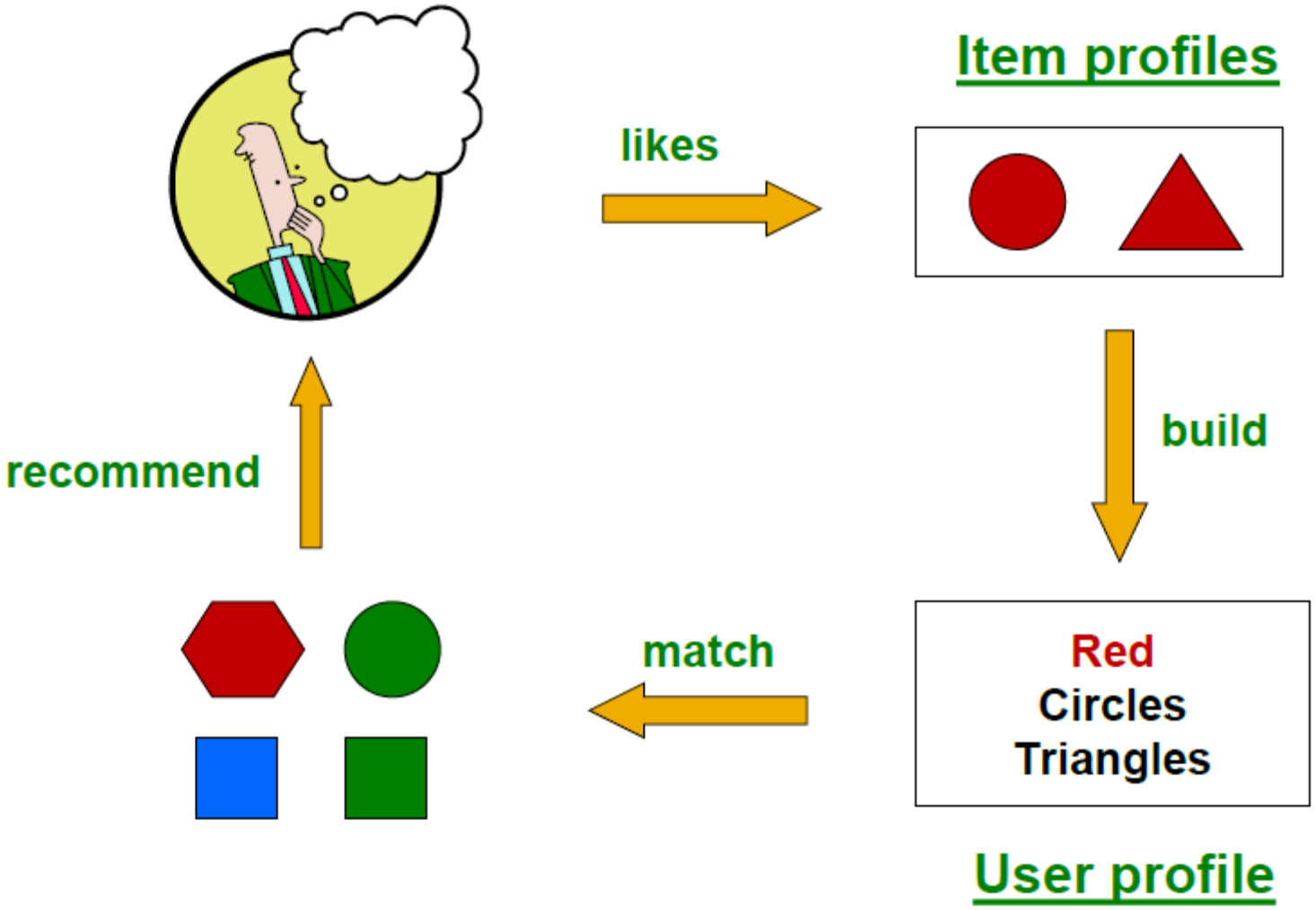
Extrapolating utilities

- **Key problem: Utility matrix *U* is sparse**
  - Most people have not rated most items
  - **Cold start:**
    - New items have no ratings
    - New users have no history
- **Three approaches to recommender systems:**
  - Content-based
  - Collaborative
  - Latent factor based

Content-based Recommendations

- **<u>Main idea:</u> Recommend items to customer *x* similar to previous items rated highly by *x***
- ***Example:***
  - Movie recommendations
    - Recommend movies with **same actor(s), director, genre**, …
  - Websites, blogs, news
    - Recommend other sites with **"similar" content**

Plan of action



Item profiles

- For each item, create an **item profile**
- Profile is a set (vector) of features
  - Movies: author, title, actor, director,…
  - Text: Set of "important" words in document
- How to pick important features?
- Usual heuristic from text mining is TF-IDF (Term frequency * Inverse Doc Frequency)
  - Term … Feature
  - Document … Item

TF - IDF

- f_{ij} = frequency of term (feature) i in doc (item) j

$$TF_{ij} = \frac{f_{ij}}{\max_k f_{kj}}$$

- n_i = number of docs that mention term i
- N = total number of docs

$$IDF_i = \log \frac{N}{n_i}$$

- TF-IDF score: $w_{ij} = TF_{ij} \times IDF_i$
- Doc profile = set of words with highest TF-IDF scores, together with their scores

User profiles and prediction

- **User profile possibilities:**
  - Weighted average of rated item profiles
  - Variation: weight by difference from average rating for item
- …
- **Prediction heuristic:**
  - Given user profile x and item profile i, estimate

$$u(\vec{x},\vec{i}) = \cos(\vec{x},\vec{i}) = (\vec{x}\cdot\vec{i}) / (|\vec{x}| \cdot |\vec{i}|)$$

Pros: content-based approach

- No need for data on other users
  - No cold-start or sparsity problems
- Able to recommend to users with unique tastes
- Able to recommend new & unpopular items
- No first-rater problem
- Able to provide explanations
  - Can provide explanations of recommended items by listing content-features that caused an item to be recommended

Cons: Content-based approach

- Finding the appropriate features is hard
  - E.g., images, movies, music
- Recommendations for new users
  - How to build a user profile?
- Overspecialization
  - Never recommends items outside user's content profile
  - People might have multiple interests
  - Unable to exploit quality judgments of other users

Exercise

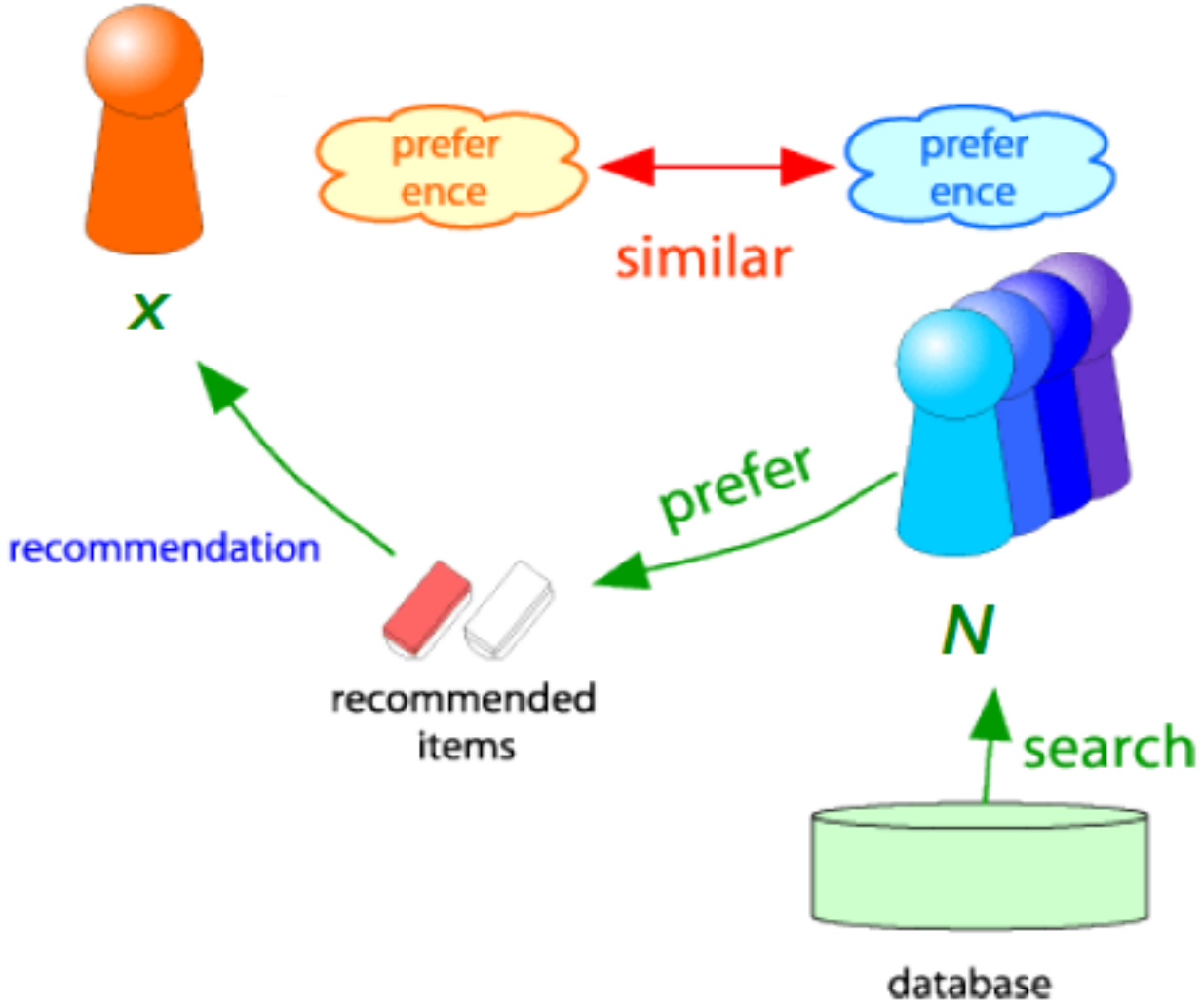
- Three computers, A, B, and C, have the numerical features listed below:

Feature	A	B	C
Processor speed	3	2.5	2.8
Disk size	500	320	640
Main-memory size	6	4	6

- Compute the cosines of the angles between the vectors for each pair of the three computers
- A certain user has rated the three computers as follows: A: 4 stars, B: 2 stars, C: 5 stars
  - Normalize the ratings for this user
  - Compute a user profile for the user, with components for processor speed, disk size, and main memory size

Collaborative filtering

- Consider user ***x***
- Find set ***N*** of other users whose ratings are "similar" to x's ratings
- Estimate ***x's*** ratings based on ratings of users in ***N***



Finding "similar" users

- Let r_x be the vector of user x's ratings
- **Jaccard similarity measure**
  - Problem: Ignores the value of the rating

r_x, r_y as sets:
$r_x = \{1, 4, 5\}$
$r_y = \{1, 3, 4\}$

- **Cosine similarity measure**

$$\text{sim}(x, y) = \cos(r_x, r_y) = \frac{r_x \cdot r_y}{||r_x|| \cdot ||r_y||}$$

r_x, r_y as points:
$r_x = \{1, 0, 0, 1, 3\}$
$r_y = \{1, 0, 2, 2, 0\}$

  - Problem: Treats missing ratings as "negative/dislike"
- **Pearson correlation coefficient**
  - S_{xy} = items rated by both users x and y

$$sim(x, y) = \frac{\sum_{s \in S_{xy}} (r_{xs} - \overline{r_x})(r_{ys} - \overline{r_y})}{\sqrt{\sum_{s \in S_{xy}} (r_{xs} - \overline{r_x})^2} \sqrt{\sum_{s \in S_{xy}} (r_{ys} - \overline{r_y})^2}}$$

$\overline{r_x}$, $\overline{r_y}$ … avg. rating of x, y

Similarity metric

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

- **Intuitively we want: sim(*A, B) > sim(A, C)***
- **Jaccard similarity: 1/5 < 2/4**
- **Cosine similarity: 0.386 > 0.322**
  - Considers missing ratings as "negative"
  - **Solution:**
    - **subtract the (row) mean**
    - **redefine the range of rating: 0 (low rating) or 1 (high rating)**

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	2/3			5/3	−7/3		
B	1/3	1/3	−2/3				
C				−5/3	1/3	4/3	
D		0					0

sim A,B vs. A,C:
0.092 > -0.559

Rating predictions

- **From similarity metric to recommendations:**
- Let r_x be the vector of user ***x's*** ratings
- Let ***N*** be the set of ***k*** users most similar to ***x*** who have rated item ***i***
- **Prediction for item *s of user x:***
  - Means of k ratings for item i $$r_{xi} = \frac{1}{k} \sum_{y \in N} r_{yi}$$
  - Weighted Means of k ratings for item i $$r_{xi} = \frac{\sum_{y \in N} s_{xy} \cdot r_{yi}}{\sum_{y \in N} s_{xy}}$$
    - $s_{xy} = sim(x,y)$
- **Many other tricks possible…**

Item-item collaborative filtering

- For item ***i, find other similar items***
- Estimate rating for item ***i*** based on ratings for similar items
- Can use same similarity metrics and prediction functions as in user-user model

$$r_{xi} = \frac{\sum_{j \in N(i;x)} s_{ij} \cdot r_{xj}}{\sum_{j \in N(i;x)} s_{ij}}$$

s_{ij}… similarity of items ***i*** and ***j***
r_{xj}…rating of user ***u*** on item ***j***
$N(i;x)$… set items rated by ***x*** similar to ***i***

Example

users

movies	1	2	3	4	5	6	7	8	9	10	11	12
1	1		3			5			5		4	
2			5	4			4			2	1	3
3	2	4		1	2		3		4	3	5	
4		2	4		5			4			2	
5			4	3	4	2					2	5
6	1		3		3			2			4	

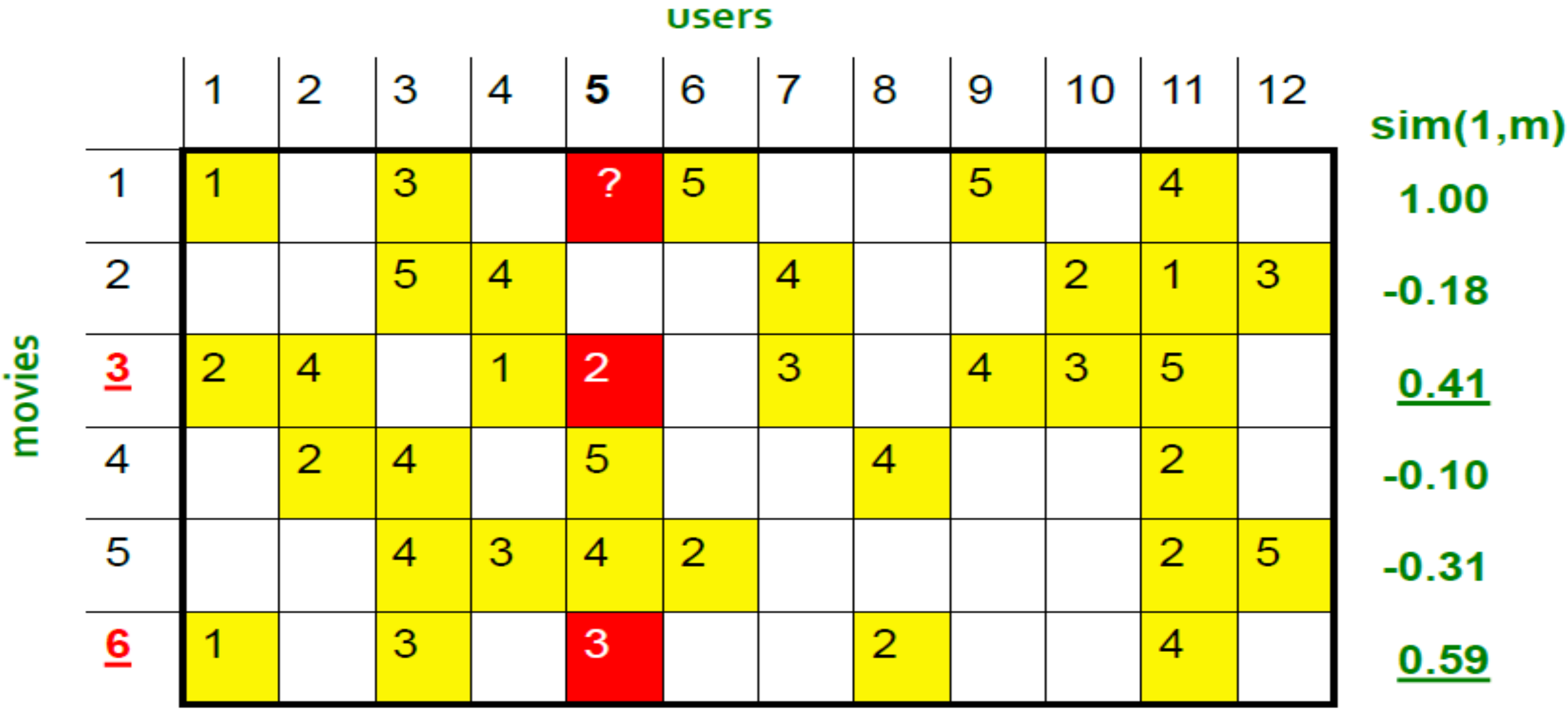
- unknown rating

- rating between 1 to 5

users

movies	1	2	3	4	**5**	6	7	8	9	10	11	12
1	1		3		?	5			5		4	
2			5	4			4			2	1	3
3	2	4		1	2		3		4	3	5	
4		2	4		5			4			2	
5			4	3	4	2					2	5
6	1		3		3			2			4	

- estimate rating of movie **1** by user **5**



Neighbor selection:
Identify movies similar to movie **1**, **rated by user 5**

Compute similarity weights:
$s_{1,3}=0.41$, $s_{1,6}=0.59$

users

movies

	1	2	3	4	**5**	6	7	8	9	10	11	12	sim(1,m)
1	1		3		?	5			5		4		1.00
2			5	4			4			2	1	3	-0.18
3	2	4		1	2		3		4	3	5		**0.41**
4		2	4		5			4			2		-0.10
5			4	3	4	2					2	5	-0.31
6	1		3		3			2			4		**0.59**

Predict by taking weighted average:

$r_{1.5}$ = (0.41*2 + 0.59*3) / (0.41+0.59) = 2.6

$$r_{ix} = \frac{\sum_{j \in N(i;x)} s_{ij} \cdot r_{jx}}{\sum s_{ij}}$$

Common practice

- Define similarity s_{ij} of items ***i*** and ***j***
- Select ***k*** nearest neighbors ***N(i; x)***
  - ***Items most similar to i, that were rated by x***
- Estimate rating r_{xi} ***as the weighted average:***

$$r_{xi} = b_{xi} + \frac{\sum_{j \in N(i;x)} s_{ij} \cdot (r_{xj} - b_{xj})}{\sum_{j \in N(i;x)} s_{ij}}$$

baseline estimate for r_{xi}

$$b_{xi} = \mu + b_x + b_i$$

- μ = overall mean movie rating
- b_x = rating deviation of user ***x***
  = (*avg. rating of user* ***x***) $- \mu$
- b_i = rating deviation of movie ***i***

Item-item vs. User-user

	Avatar	LOTR	Matrix	Pirates
Alice	1		0.8	
Bob		0.5		0.3
Carol	0.9		1	0.8
David			1	0.4

- **In practice, it has been observed that item-item often works better than user-user**
- **Why? Items are simpler, users have multiple tastes**

Pros/Cons of Collaborative Filtering

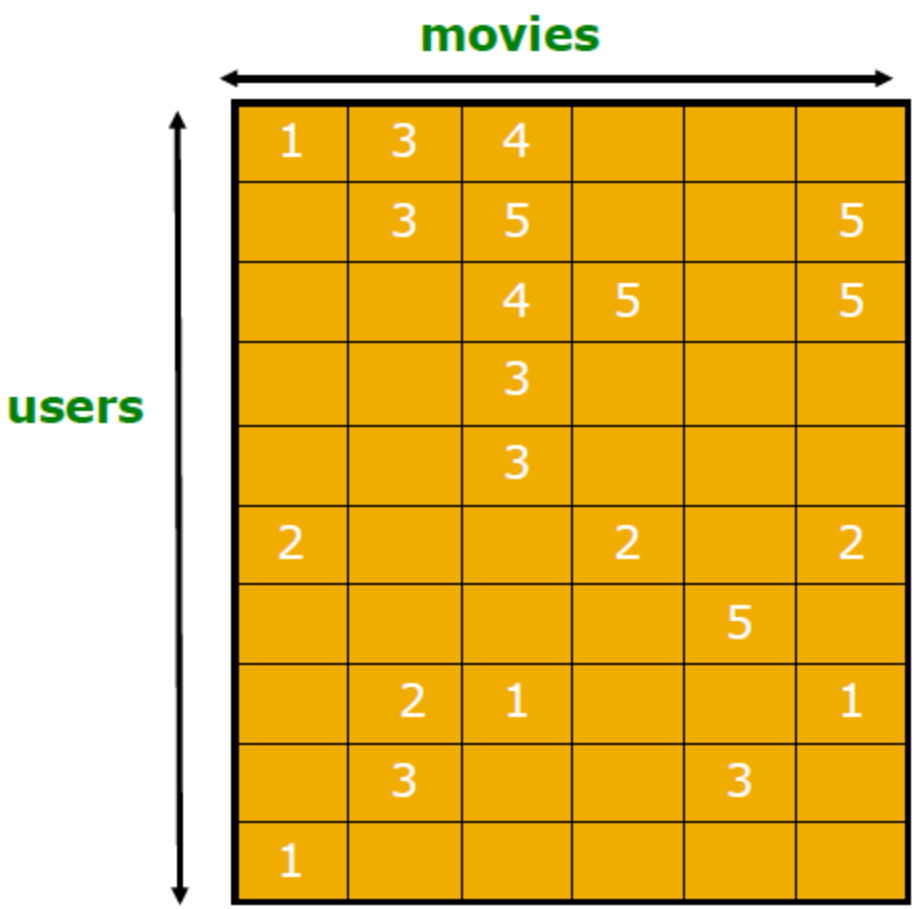
- Cold Start: ☹
  - Need enough users in the system to find a match
- Sparsity: ☹
  - The user/ratings matrix is sparse
  - Hard to find users that have rated the same items
- First rater: ☹
  - Cannot recommend an item that has not been previously rated
  - New items, Esoteric items
- Popularity bias: ☹
  - Cannot recommend items to someone with unique taste
  - Tends to recommend popular items

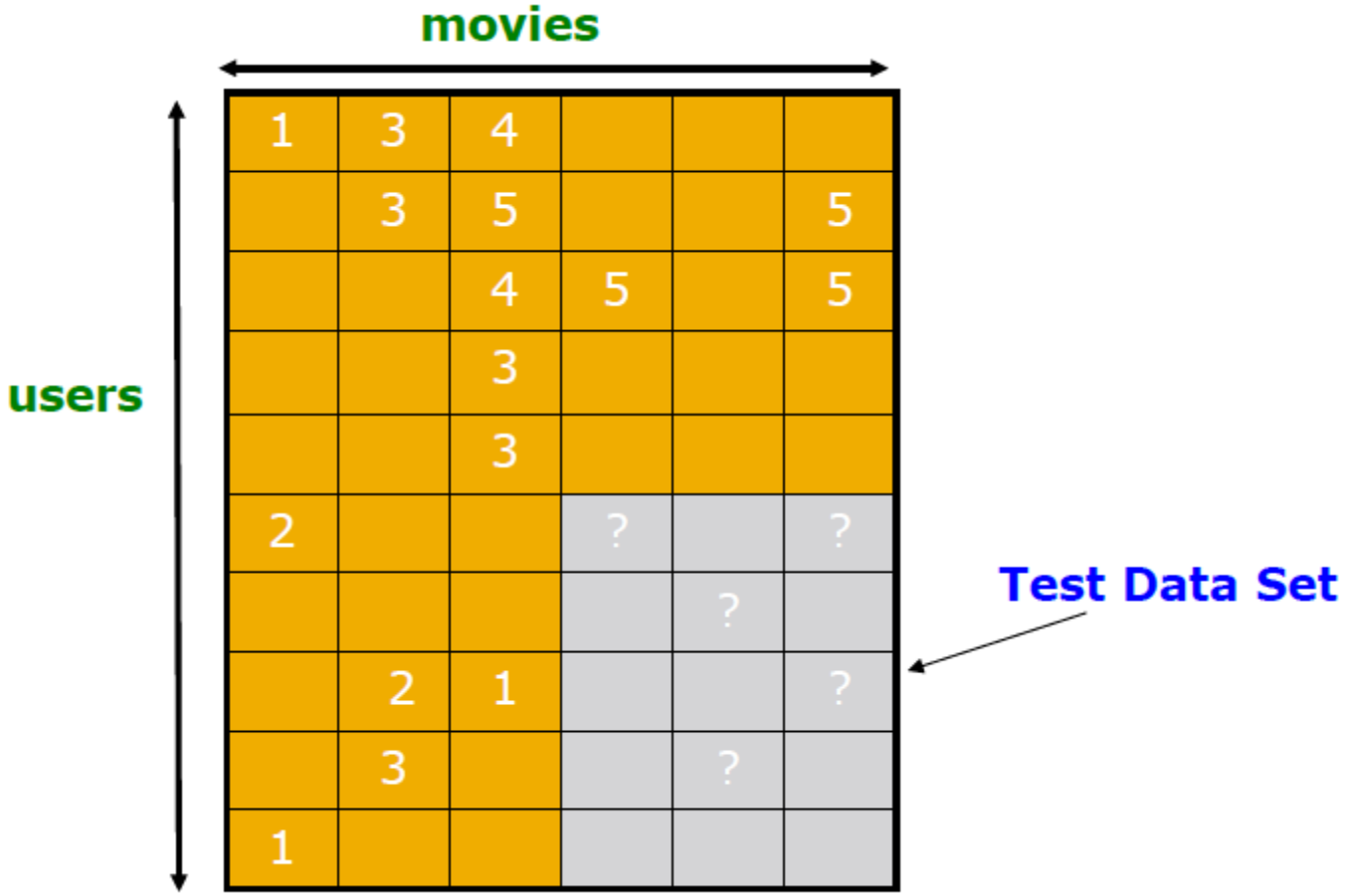
- Works for any kind of item ☺
  - No feature selection needed

Hybrid method

- Implement two or more different recommenders and combine predictions
- Add content-based methods to collaborative filtering
  - Item profiles for new item problem
  - Demographics to deal with new user problem

Evaluation


movies
users
1 3 4
3 5 5
4 5 5
3
3
2 2 2
5
2 1 1
3 3
1


movies
users
1 3 4
3 5 5
4 5 5
3
3
2 ? ?
? ?
2 1 ?
3 ?
1
Test Data Set

Evaluation predictions

- **Compare predictions with known ratings**
  - **Root-mean-square error** (RMSE)
    - $\sqrt{\sum_{xi}(r_{xi} - r^*_{xi})^2}$ where r_{xi} is predicted, r^*_{xi} is the true rating of x on i
  - **Precision at top 10**:
    - % of those in top 10
  - **Rank Correlation**:
    - Spearman's *correlation* between system's and user's complete rankings

Spearman's *correlation, if* if all *n* ranks are *distinct integers*

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

d_i: difference ranks between system and user
n: number of ranks

Another approach: 0/1 model

Coverage:
Number of items/users for which system can make predictions

Precision:
Accuracy of predictions

Receiver operating characteristic (ROC)
Tradeoff curve between false positives and false negatives

Complexity

- Expensive step is finding ***k*** most similar customers: ***O(|X|)***
- Too expensive to do at runtime: Could pre-compute
- Naïve pre-computation takes time **O(k ·|X|)**
  - **X … set of customers**
- **We already know how to do this!**
  - Near-neighbor search in high dimensions (LSH)
  - Clustering
  - Dimensionality reduction

The Netflix Prize

- Training data
  - 100 million ratings (over 8 billions ratings), 480,000 users, 17,770 movies
  - 6 years of data: 2000-2005
- Test data
  - Last few ratings of each user (2.8 million)
  - Evaluation criterion: Root Mean Square Error (RMSE)

$$\frac{1}{|R|}\sqrt{\sum_{(i,x)\in R}(\hat{r}_{xi} - r_{xi})^2}$$

  - Netflix's system RMSE: 0.9514
- Competition
  - 2,700+ teams
  - $1 million prize for 10% improvement on Netflix

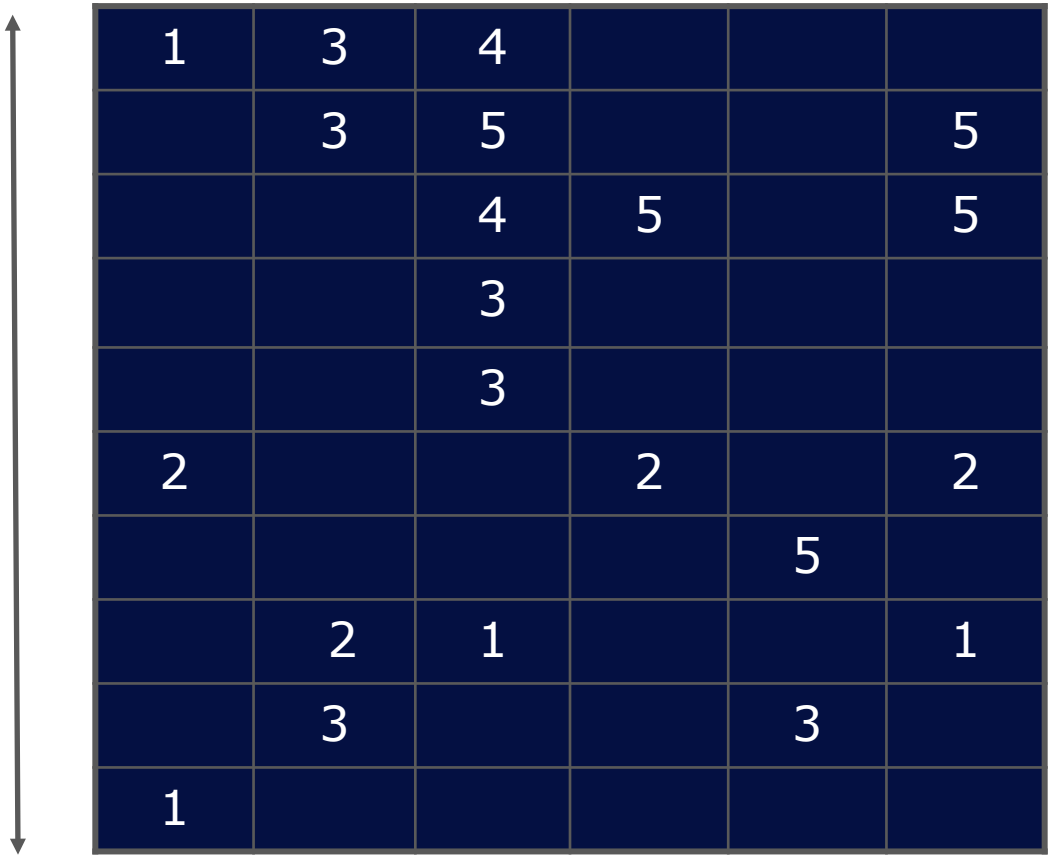
Latent factor model

The Netflix Utility Matrix *R*

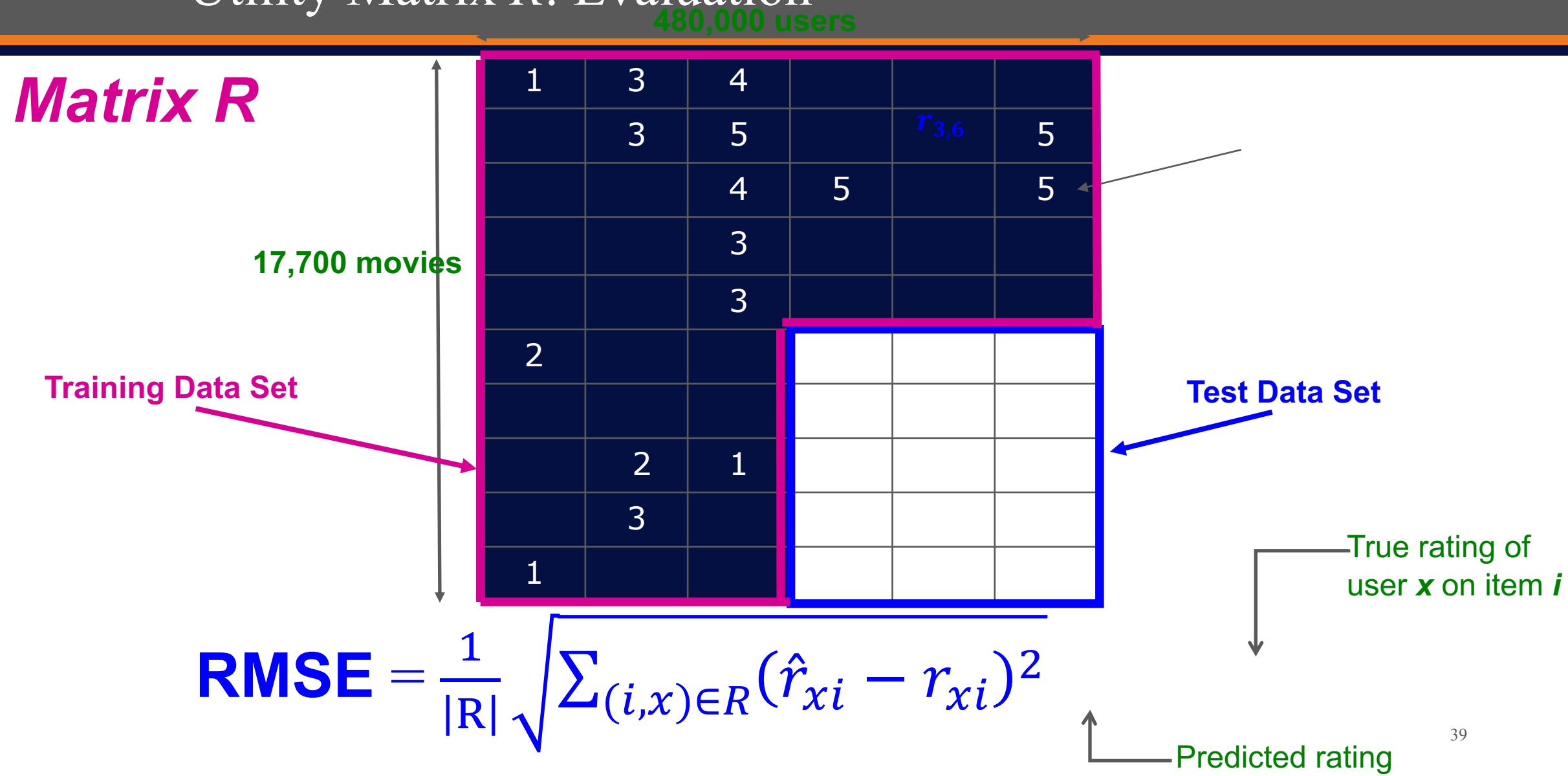
Matrix R

480,000 users

17,700 movies



Utility Matrix R: Evaluation



$$\textbf{RMSE} = \frac{1}{|R|}\sqrt{\sum_{(i,x)\in R}(\hat{r}_{xi} - r_{xi})^2}$$

Recall: Collaborative Filtering (CF)

- **Earliest and most popular collaborative filtering method**
- Derive unknown ratings from those of "**similar**" movies (item-item variant)
- Define **similarity measure** s_{ij} of items ***i*** and ***j***
- Select ***k***-nearest neighbors, compute the rating
  - ***N(i; x):*** items most similar to ***i*** that were rated by ***x***

$$\hat{r}_{xi} = \frac{\sum_{j \in N(i;x)} s_{ij} \cdot r_{xj}}{\sum_{j \in N(i;x)} s_{ij}}$$

s_{ij}… similarity of items ***i*** and ***j***
r_{xj}…rating of user ***x*** on item ***j***
N(i;x)… set of items similar to item ***i*** that were rated by ***x***

Modeling Local & Global Effects

- **Global:**
  - Mean movie rating: **3.7 stars**
  - *The Sixth Sense* is **0.5** stars above avg.
  - Joe rates **0.2** stars below avg.

  ⇒ Baseline estimation: *Joe* will rate *The Sixth Sense* 4 stars

- **Local neighborhood (CF/NN):**
  - *Joe* didn't like related movie *Signs*
  - *Rated it 1 stars below avg.*

    ⇒ Final estimate: *Joe* will rate *The Sixth Sense* (4-1) = 3 stars

Modeling Local & Global Effects

- **In practice we get better estimates if we model deviations:**

$$\hat{r}_{xi} = b_{xi} + \frac{\sum_{j \in N(i;x)} s_{ij} \cdot (r_{xj} - b_{xj})}{\sum_{j \in N(i;x)} s_{ij}}$$

baseline estimate for r_{xi}

$$b_{xi} = \mu + b_x + b_i$$

μ = overall mean rating
b_x = rating deviation of user x
= (*avg. rating of* user x) – μ
b_i = (*avg. rating of* movie i) – μ

Problems/Issues:

1) Similarity measures are "arbitrary"

2) Pairwise similarities neglect interdependencies among users

3) Taking a weighted average can be restricting

Solution: Instead of s_{ij} use w_{ij} that we estimate directly from data

Idea: Interpolation Weights w_{ij}

- Use a **weighted sum** rather than **weighted avg.**:

$$\widehat{r_{xi}} = b_{xi} + \sum_{j \in N(i;x)} w_{ij}\left(r_{xj} - b_{xj}\right)$$

- **A few notes:**
  - $\boldsymbol{N(i;x)}$ … set of movies rated by user $\boldsymbol{x}$ that are similar to movie $\boldsymbol{i}$
  - $\boldsymbol{w_{ij}}$ is the interpolation weight (some real number)
    - We allow: $\boldsymbol{\sum_{j \in N(i,x)} w_{ij} \neq 1}$
  - $\boldsymbol{w_{ij}}$ models interaction between pairs of movies
    (it does not depend on user $\boldsymbol{x}$)

Idea: Interpolation Weights w_{ij}

- $\widehat{r_{xi}} = b_{xi} + \sum_{j \in N(i,x)} w_{ij}\left(r_{xj} - b_{xj}\right)$
- **How to set w_{ij}?**
  - Remember, error metric is: $\frac{1}{|R|}\sqrt{\sum_{(i,x)\in R}(\hat{r}_{xi} - r_{xi})^2}$ or equivalently **SSE:** $\sum_{(i,x)\in R}(\hat{r}_{xi} - r_{xi})^2$
  - Find w_{ij} that minimize **SSE** on **training data!**
    - Models relationships between item ***i*** and its neighbors ***j***
  - w_{ij} can be **learned/estimated** based on ***x*** and all other users that rated ***i***

Why is this a good idea?

Recommendations via optimization

- **Goal: Make good recommendations**
  - Quantify goodness using **RMSE:**

    Lower RMSE ⇒ better recommendations
  - Want to make good recommendations on items that user has not yet seen. Can't really do this!
  - Let's set build a system such that it works well on known (user, item) ratings
  - And hope the system will also predict well the unknown ratings

Recommendations via Optimization

- **Idea: Let's set values *w* such that they work well on known (user, item) ratings**
- **How to find such values *w*?**
- **Idea:** Define an objective function and solve the optimization problem

- Find w_{ij} that minimize **SSE** on **training data**!

$$J(w) = \sum_{x,i} \left(\left[b_{xi} + \sum_{j \in N(i;x)} w_{ij} \left(r_{xj} - b_{xj} \right) \right] - r_{xi} \right)^2$$

Predicted rating — True rating

- Think of ***w*** as a vector of numbers

Interpolation Weights

- **We have the optimization problem, now what?**

$$J(w) = \sum_x \left(\left[b_{xi} + \sum_{j \in N(i;x)} w_{ij} (r_{xj} - b_{xj}) \right] - r_{xi} \right)^2$$

- **Gradient decent:**
  - **Iterate until convergence:** $\boldsymbol{w \leftarrow w - \eta \nabla_w J}$
  - **where** $\boldsymbol{\nabla_w J}$ **is the gradient (derivative evaluated on data):**

η … learning rate

$$\nabla_w J = \left[\frac{\partial J(w)}{\partial w_{ij}} \right]$$

$$= 2 \sum_{x,i} \left(\left[b_{xi} + \sum_{k \in N(i;x)} w_{ik} (r_{xk} - b_{xk}) \right] - r_{xi} \right) (r_{xj} - b_{xj})$$

for $\boldsymbol{j \in \{N(i;x), \forall i, \forall x\}}$

else $\frac{\partial J(w)}{\partial w_{ij}} = \mathbf{0}$

while $|w_{new} - w_{old}| > \varepsilon$:

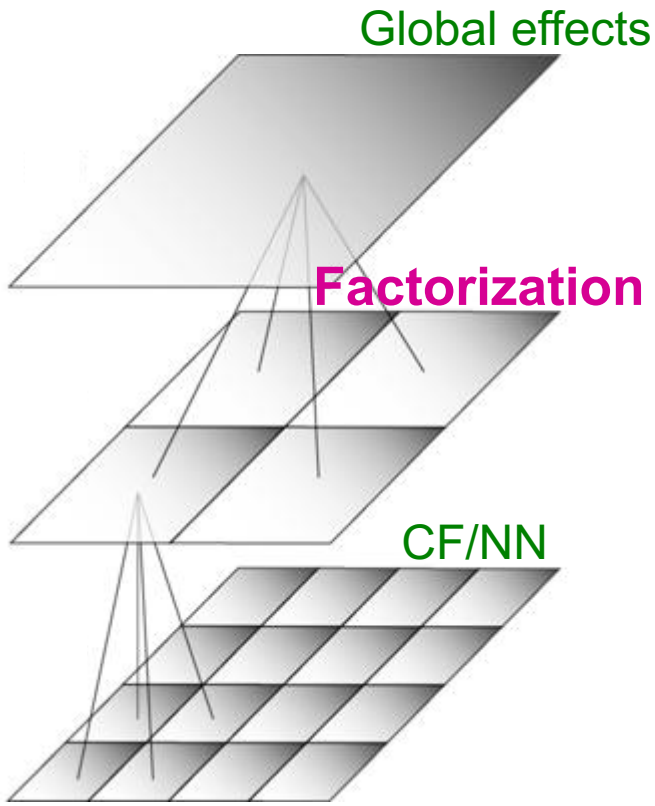
$w_{old} = w_{new}$

$w_{new} = w_{old} - \eta \cdot \nabla w_{old}$

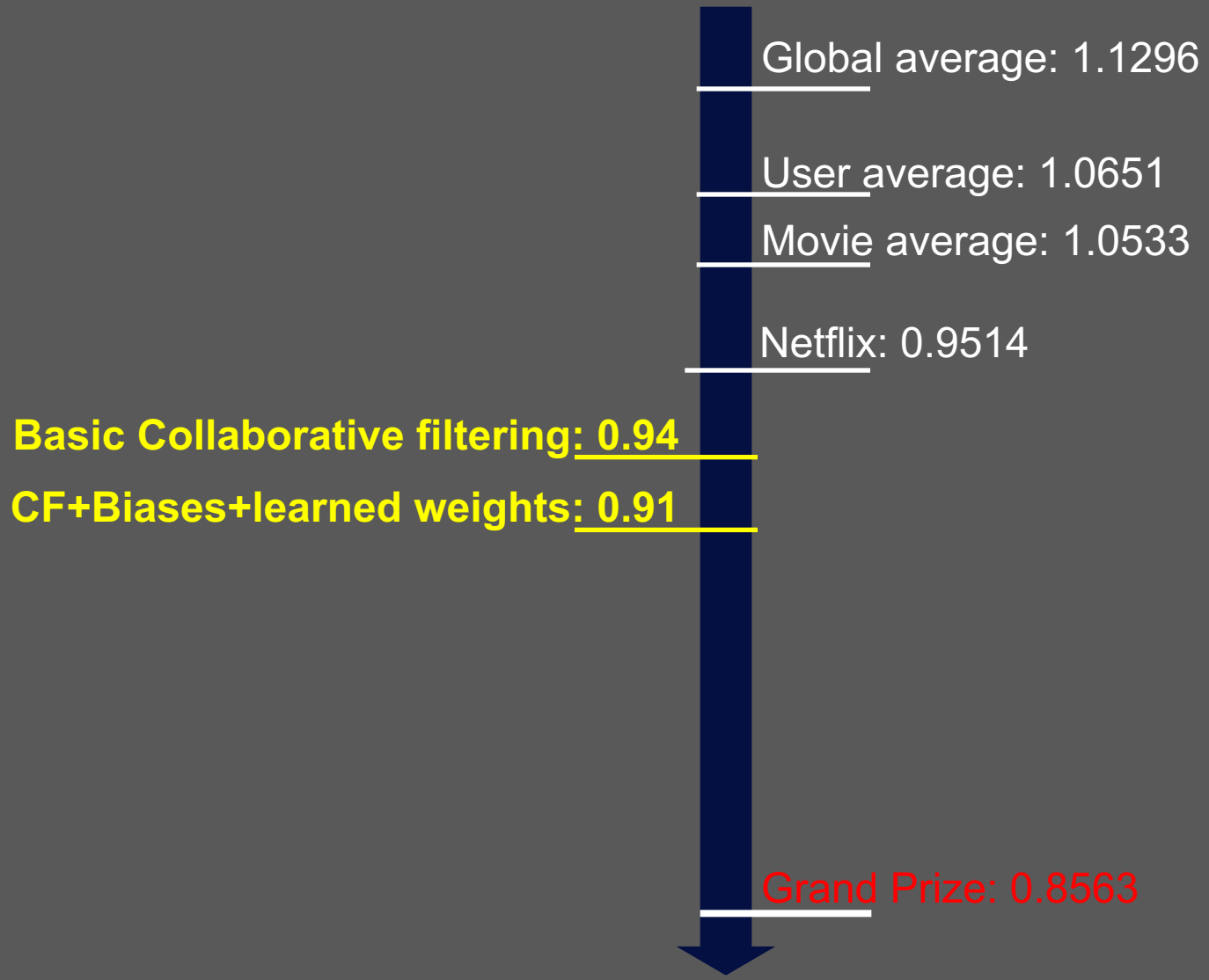
- **Note:** We fix movie $\boldsymbol{i}$, go over all $\boldsymbol{r_{xi}}$, for every movie $\boldsymbol{j \in N(i;x)}$, we compute $\frac{\partial J(w)}{\partial w_{ij}}$

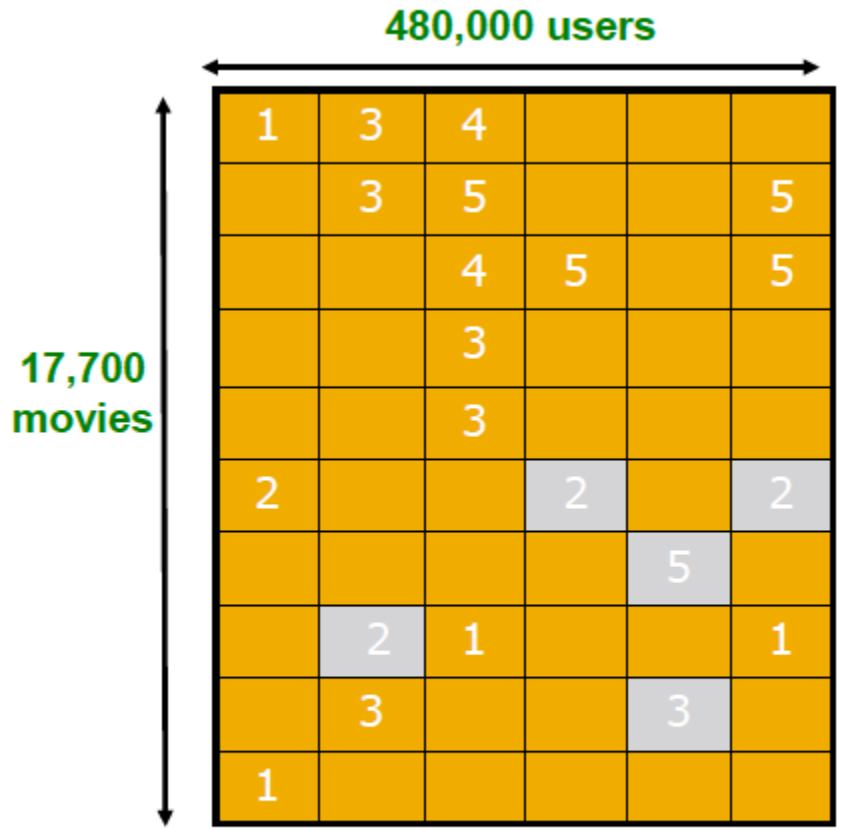
Interpolation Weights

- **So far:** $\widehat{r_{xi}} = b_{xi} + \sum_{j \in N(i;x)} w_{ij}\left(r_{xj} - b_{xj}\right)$
  - Weights w_{ij} derived based on their role; **no use of an arbitrary similarity measure** ($w_{ij} \neq s_{ij}$)
  - Explicitly account for interrelationships among the neighboring movies
- **Next: Latent factor model**
  - Extract "regional" correlations



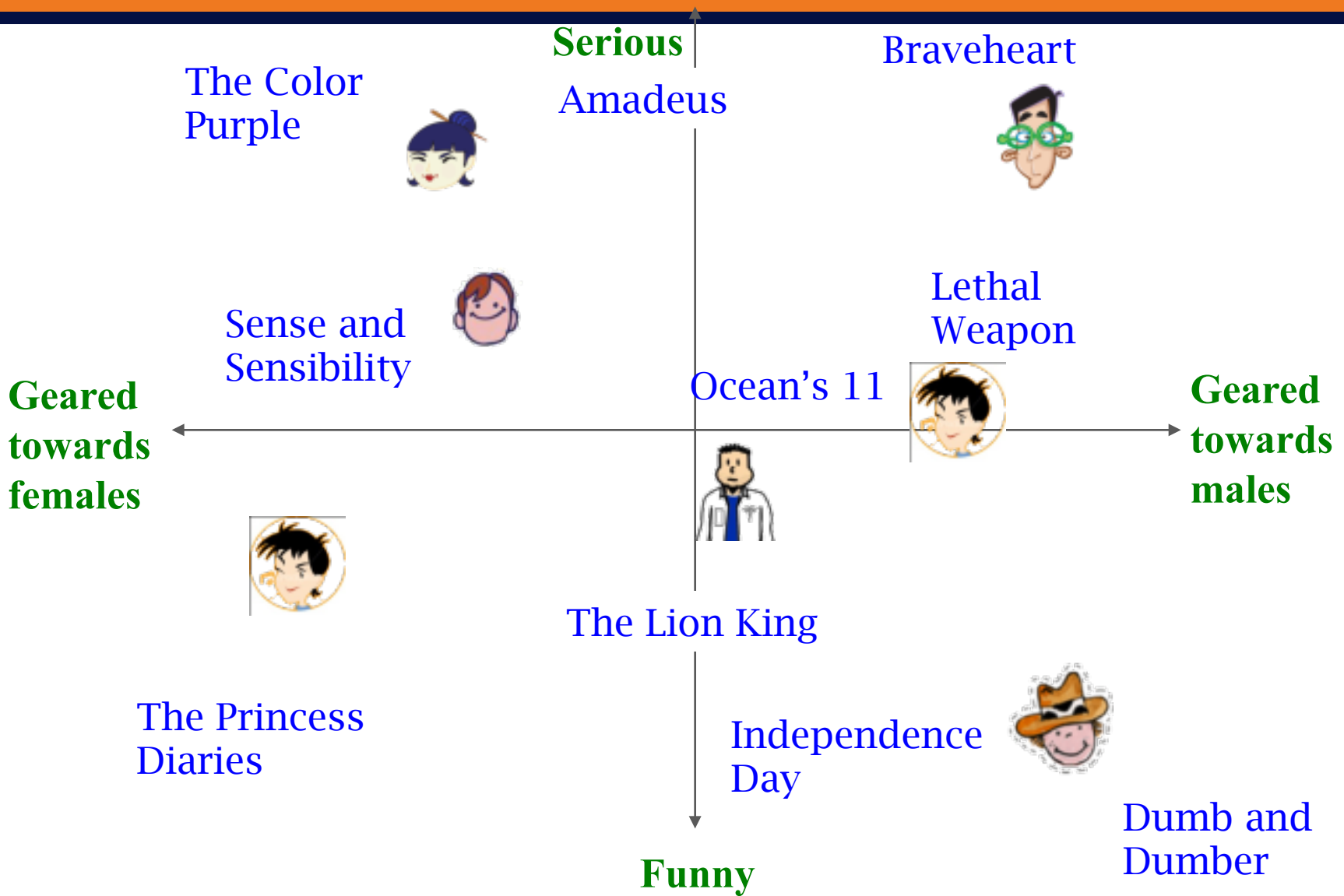
Performance of Various Methods





We want our system to predict well the hidden (known) ratings

Latent Factor Models (e.g., SVD)



Latent factors model

- "SVD" on Netflix data: $R \approx Q \cdot P^T$
- For now let's assume we can approximate the rating matrix ***R as a product of "thin"*** $Q \cdot P^T$

items

1		3			5			5		4	
		5	4			4			2	1	3
2	4		1	2		3		4	3	5	
	2	4		5			4			2	
		4	3	4	2					2	5
1		3		3			2			4	

R

≈

items

.1	-.4	.2
-.5	.6	.5
-.2	.3	.5
1.1	2.1	.3
-.7	2.1	-2
-1	.7	.3

Q

users

1.1	-.2	.3	.5	-2	-.5	.8	-.4	.3	1.4	2.4	
-.8	.7	.5	1.4	.3	-1	1.4	2.9	-.7	1.2	-.1	
2.1	-.4	.6	1.7	2.4	.9	-.3	.4	.8	.7	-.6	

factors

P^T

SVD: $A = U \Sigma V^T$

Ratings as Products of Factors

- **How to estimate the missing rating of user *x* for item *i*?**

users

items

1		3			5			5		4	
		5	4	?		4			2	1	3
2	4		1	2		3		4	3	5	
	2	4		5			4			2	
		4	3	4	2					2	5
1		3		3			2			4	

$$\hat{r}_{xi} = q_i \cdot p_x = \sum_f q_{if} \cdot p_{xf}$$

$\approx$

q_i = row i of Q

p_x = column x of P^T

items

.1	-.4	.2
-.5	.6	.5
-.2	.3	.5
1.1	2.1	.3
-.7	2.1	-2
-1	.7	.3

factors

Q

$\cdot$

users

factors

1.1	-.2	.3	.5	-2	-.5	.8	-.4	.3	1.4	2.4	-.9
-.8	.7	.5	1.4	.3	-1	1.4	2.9	-.7	1.2	-.1	1.3
2.1	-.4	.6	1.7	2.4	.9	-.3	.4	.8	.7	-.6	.1

P^T

Ratings as Products of Factors

- **How to estimate the missing rating of user *x* for item *i*?**

users

items

1		3			5			5		4	
		5	4	?		4			2	1	3
2	4		1	2		3		4	3	5	
	2	4		5			4			2	
		4	3	4	2					2	5
1		3		3			2			4	

$$\approx$$

$$\hat{r}_{xi} = q_i \cdot p_x$$

$$= \sum_f q_{if} \cdot p_{xf}$$

q_i = row i of Q
p_x = column x of P^T

items

.1	-.4	.2
-.5	**.6**	**.5**
-.2	.3	.5
1.1	2.1	.3
-.7	2.1	-2
-1	.7	.3

factors

Q

$\cdot$

users

factors

1.1	-.2	.3	.5	**-2**	-.5	.8	-.4	.3	1.4	2.4	-.9
-.8	.7	.5	1.4	**.3**	-1	1.4	2.9	-.7	1.2	-.1	1.3
2.1	-.4	.6	1.7	**2.4**	.9	-.3	.4	.8	.7	-.6	.1

P^T

Ratings as Products of Factors

- **How to estimate the missing rating of user *x* for item *i*?**

users

items

1		3			5			5		4	
		5	4	**2.4**		4			2	1	3
2	4		1	2		3		4	3	5	
	2	4		5			4			2	
		4	3	4	2					2	5
1		3		3			2			4	

$$\hat{r}_{xi} = q_i \cdot p_x = \sum_f q_{if} \cdot p_{xf}$$

q_i = row i of Q
p_x = column x of P^T

items

.1	-.4	.2
-.5	**.6**	**.5**
-.2	.3	.5
1.1	2.1	.3
-.7	2.1	-2
-1	.7	.3

Q

•

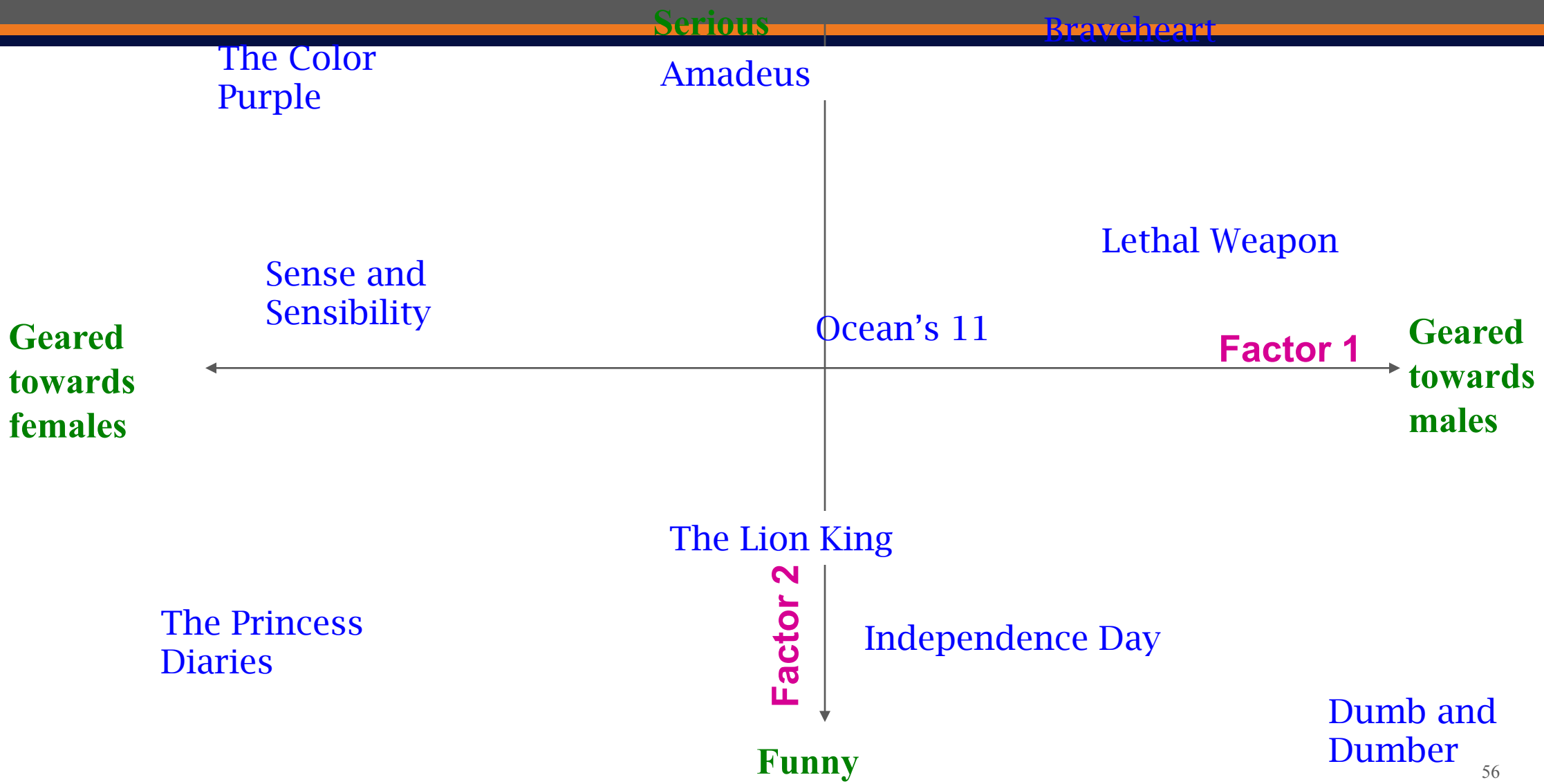
users

f factors

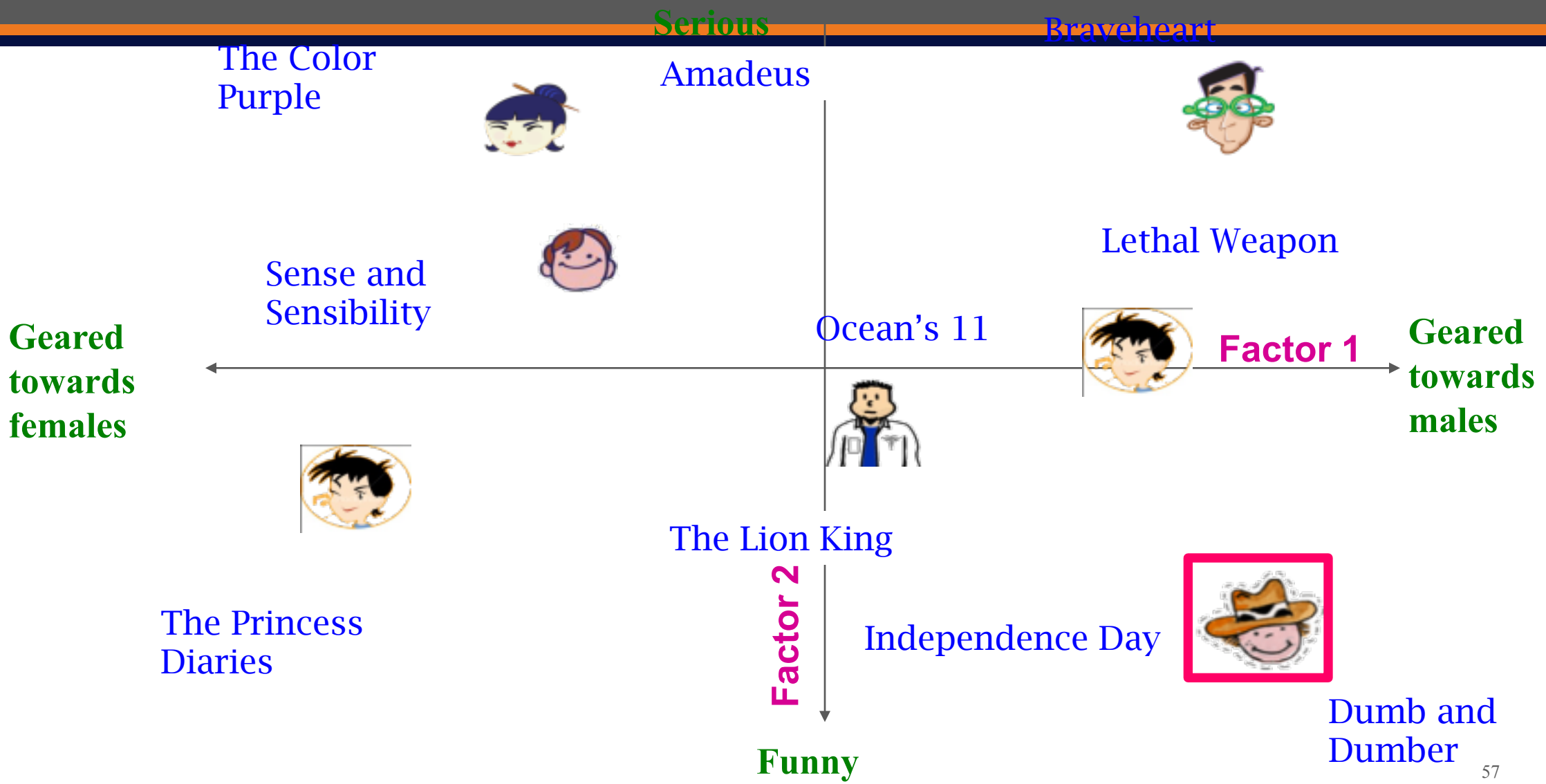
1.1	-.2	.3	.5	**-2**	-.5	.8	-.4	.3	1.4	2.4	-.9
-.8	.7	.5	1.4	**.3**	-1	1.4	2.9	-.7	1.2	-.1	1.3
2.1	-.4	.6	1.7	**2.4**	.9	-.3	.4	.8	.7	-.6	.1

P^T

Latent Factor Models

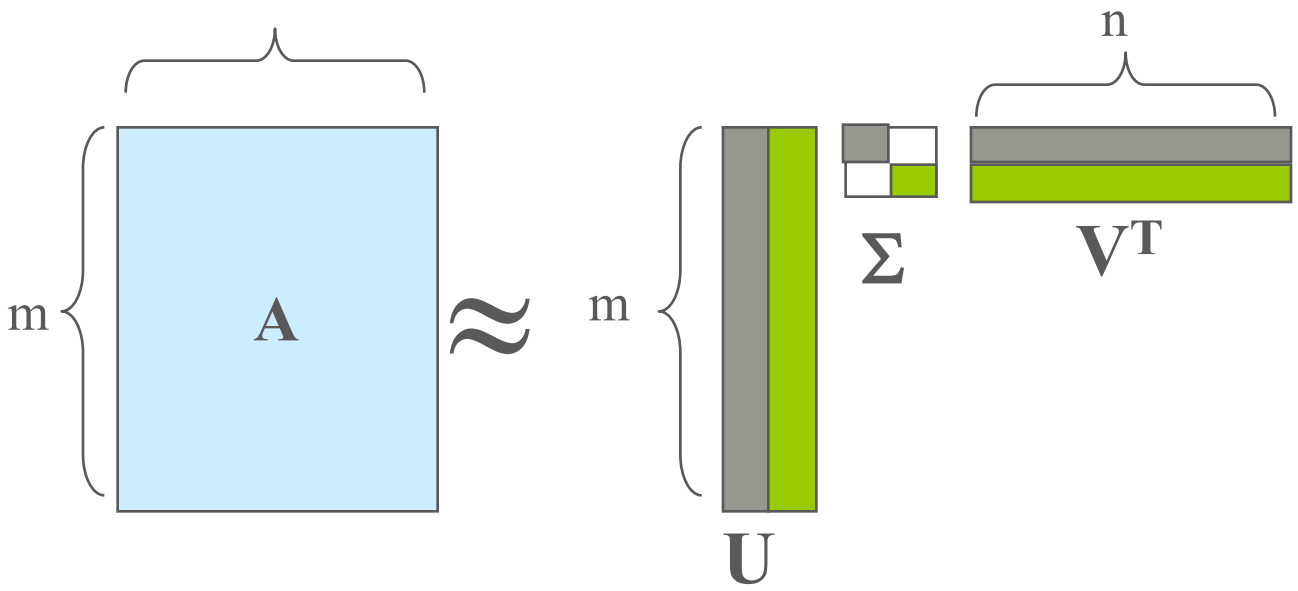


Latent Factor Models



Recap: SVD

- **Remember SVD:**
  - **A**: Input data matrix
  - **U**: Left singular vecs
  - **V**: Right singular vecs
  - **Σ**: Singular values



- **So in our case:**
  "SVD" on Netflix data: $R \approx Q \cdot P^T$
  $A = R,\ Q = U,\ P^T = \Sigma V^T$

$$\hat{r}_{xi} = q_i \cdot p_x$$

SVD: More good stuff

- We already know that SVD gives minimum reconstruction error (Sum of Squared Errors):

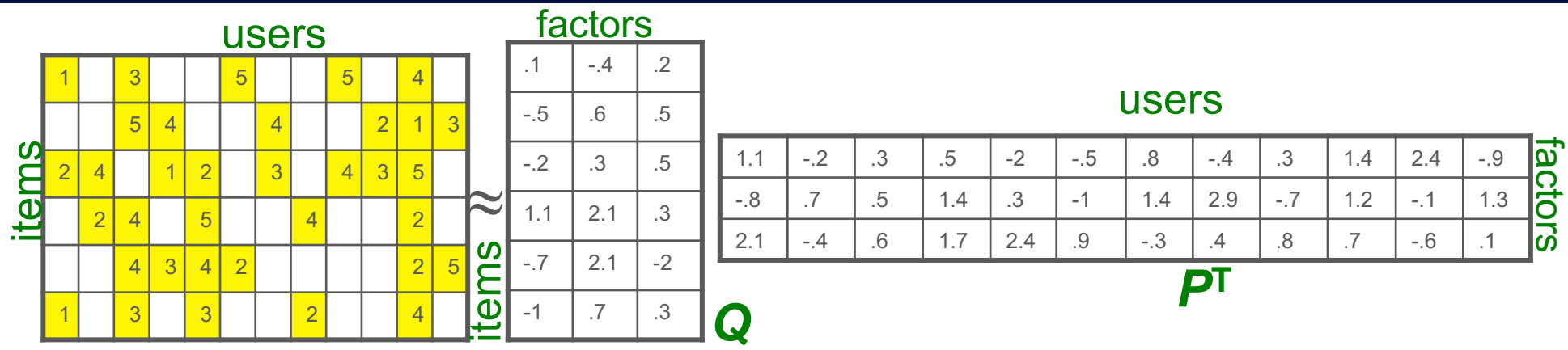
$$\min_{U,V,\Sigma} \sum_{ij \in A} \left(A_{ij} - [U\Sigma V^{\mathrm{T}}]_{ij}\right)^2$$

- Note two things:
  - SSE and RMSE are monotonically related:

$$\boldsymbol{RMSE} = \frac{\mathbf{1}}{\boldsymbol{c}}\sqrt{\boldsymbol{SSE}}$$

- **Great news: SVD is minimizing RMSE**
- Complication: The sum in SVD error term is over all entries (no-rating in interpreted as zero-rating). But our *R has missing entries*

Latent Factor Models



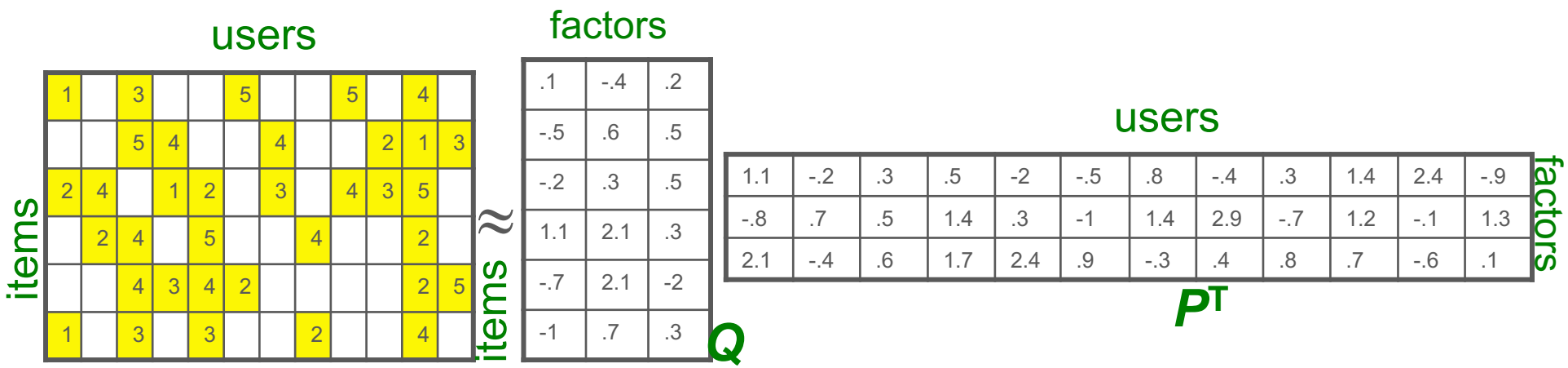
- **SVD isn't defined when entries are missing!**
- **Use specialized methods to find *P*, *Q***
  - $\min_{P,Q} \sum_{(i,x)\in R} (r_{xi} - q_i \cdot p_x)^2$
  - **Note:**
    - We don't require cols of ***P, Q*** to be orthogonal/unit length
    - ***P, Q*** map users/movies to a latent space
    - The most popular model among Netflix contestants

$$\hat{r}_{xi} = q_i \cdot p_x$$

Latent Factor Models

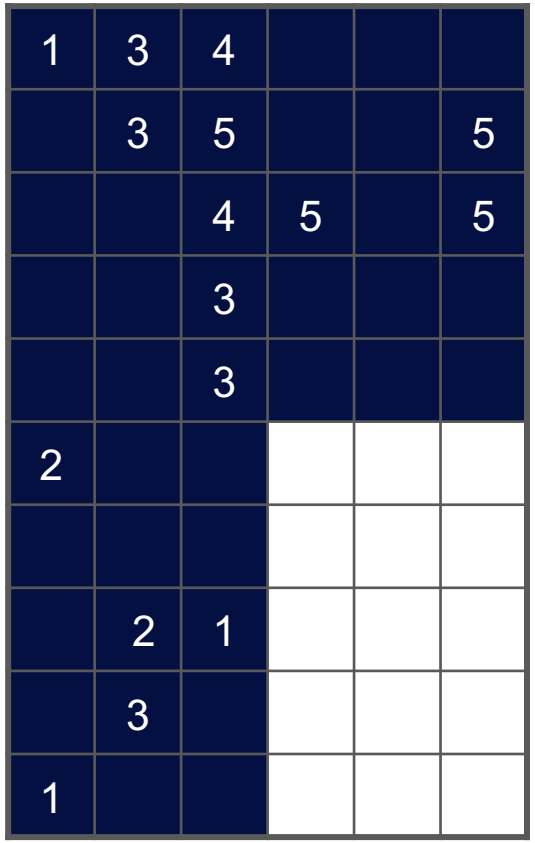
- **Our goal is to find P and Q such tat:**

$$\min_{P,Q} \sum_{(i,x)\in R} \left(r_{xi} - q_i \cdot p_x\right)^2$$

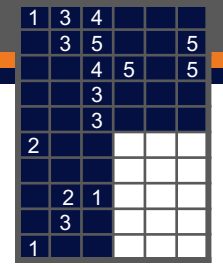


Back to Our Problem

- **Want to minimize SSE for unseen test data**
- **Idea: Minimize SSE on training data**
  - Want large ***k*** (# of factors) to capture all the signals
  - But, **SSE** on **test data** begins to rise for ***k*** > 2
- This is a classical example of **overfitting:**
  - With too much freedom (too many free parameters) the model starts fitting noise
    - That is it fits too well the training data and thus **not generalizing** well to unseen test data



Dealing with Missing Entries



- **To solve overfitting we introduce regularization:**
  - Allow rich model where there are sufficient data
  - Shrink aggressively where data are scarce

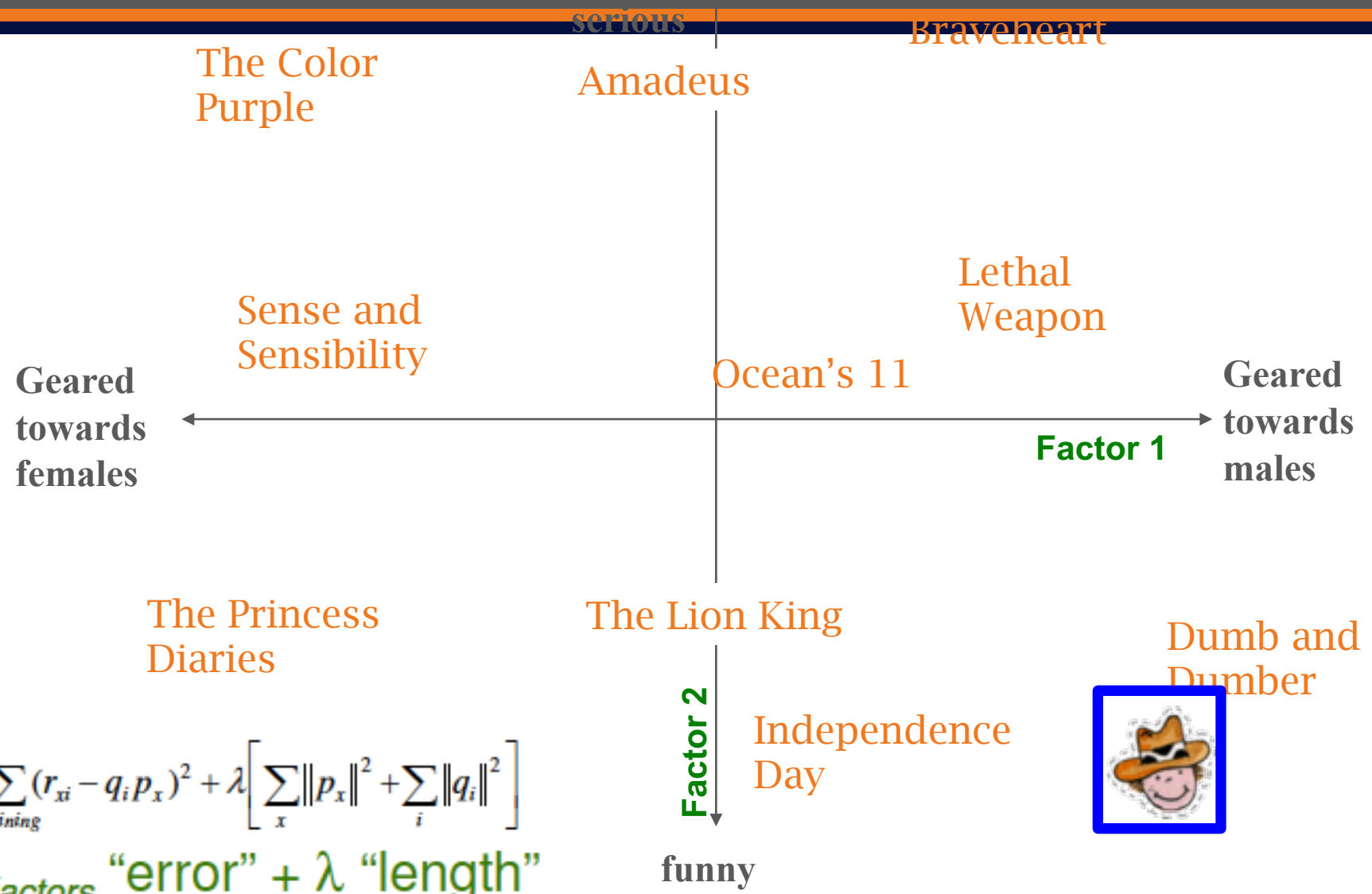
$$\min_{P,Q} \sum_{training} (r_{xi} - q_i p_x)^2 + \left[\lambda_1 \sum_x \|p_x\|^2 + \lambda_2 \sum_i \|q_i\|^2 \right]$$

"error" "length"

λ_1, λ_2 … user set regularization parameters

Note: We do not care about the "raw" value of the objective function, but we care in P,Q that achieve the minimum of the objective

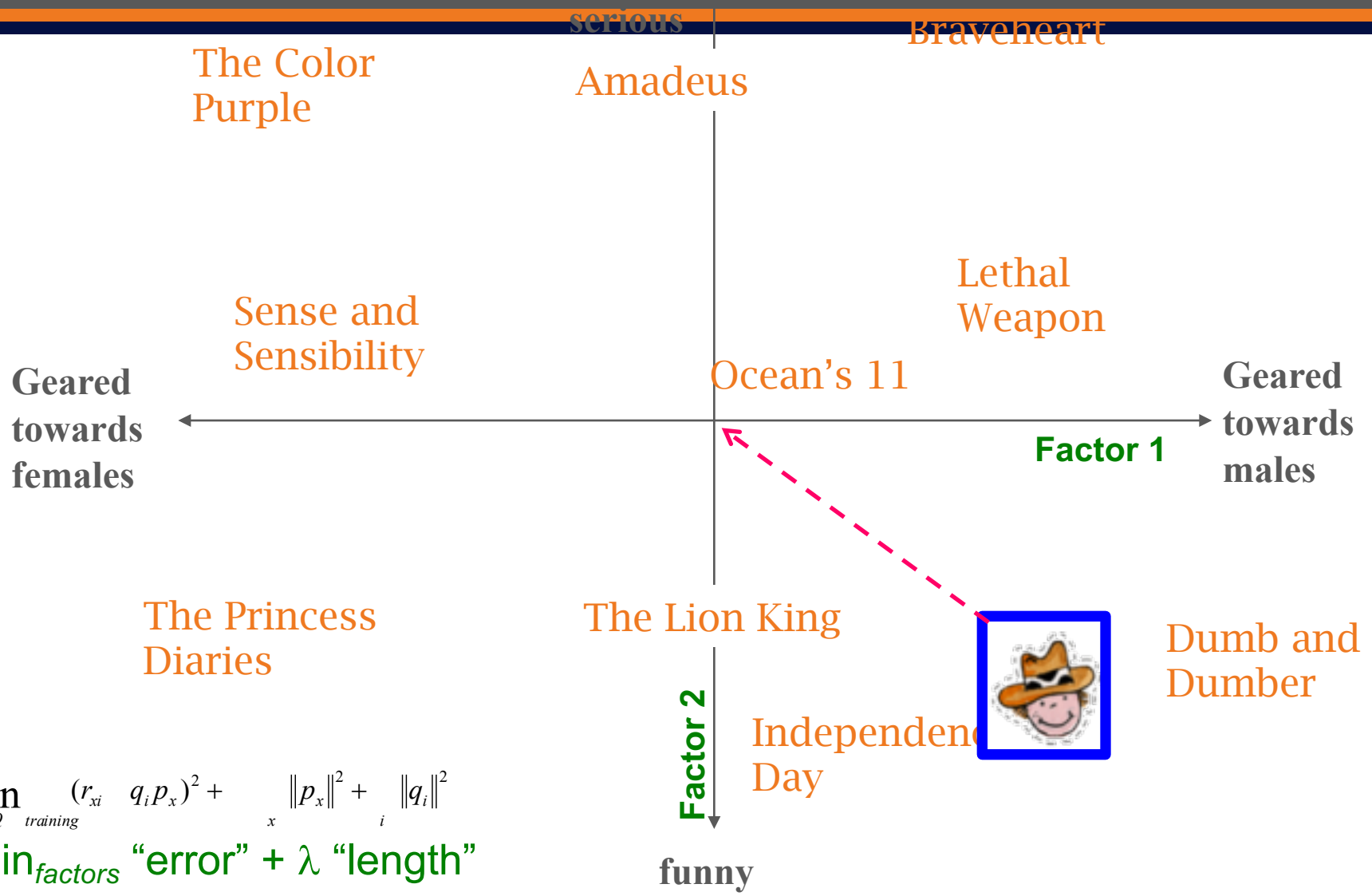
The Effect of Regularization



$$\min_{P,Q} \sum_{training} (r_{xi} - q_i p_x)^2 + \lambda \left[\sum_x \|p_x\|^2 + \sum_i \|q_i\|^2 \right]$$

$\min_{factors}$ "error" + λ "length"

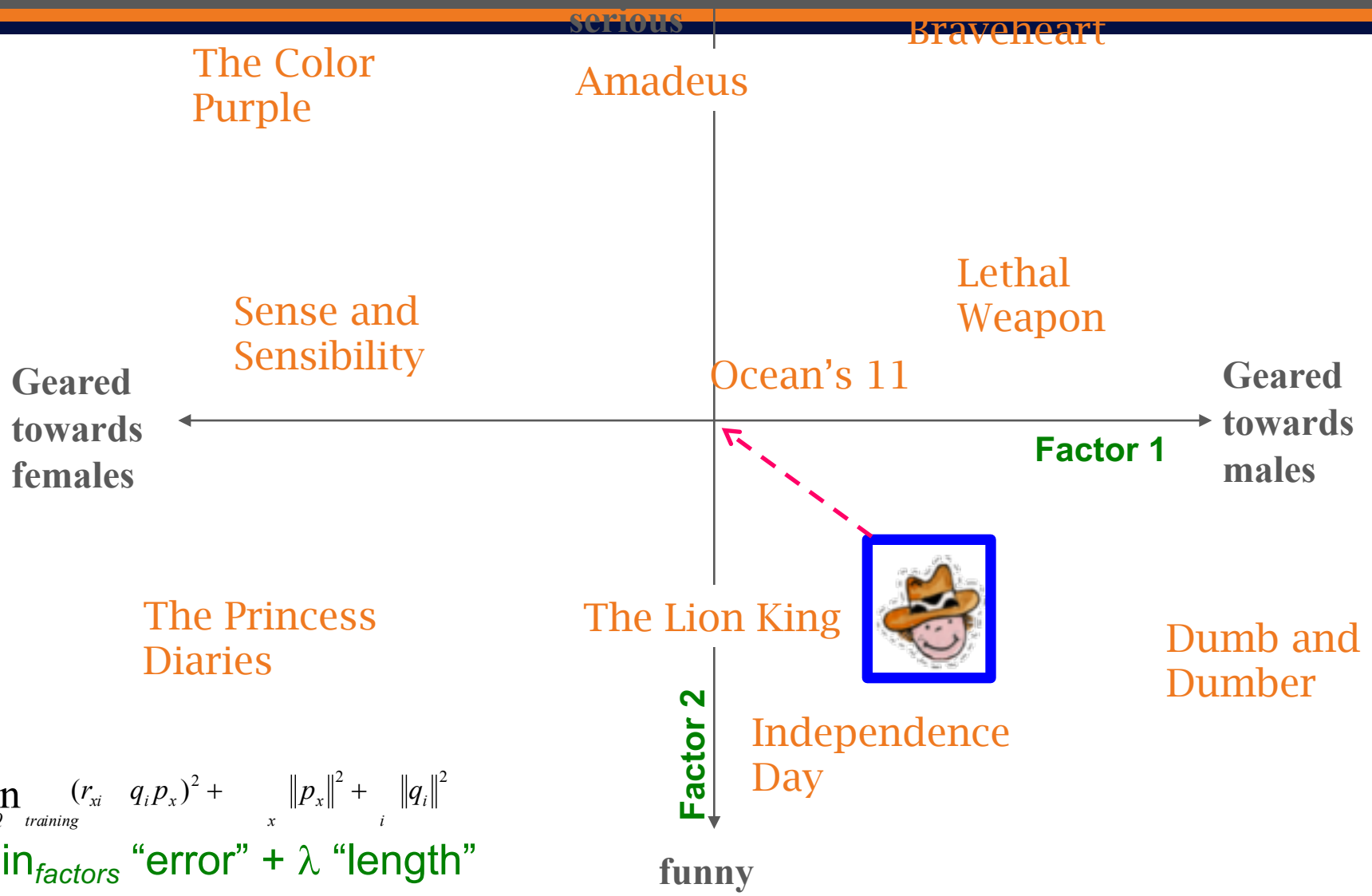
The Effect of Regularization



$$\min_{P,Q} \sum_{training} (r_{xi} - q_i p_x)^2 + \sum_x \|p_x\|^2 + \sum_i \|q_i\|^2$$

$\min_{factors}$ "error" + λ "length"

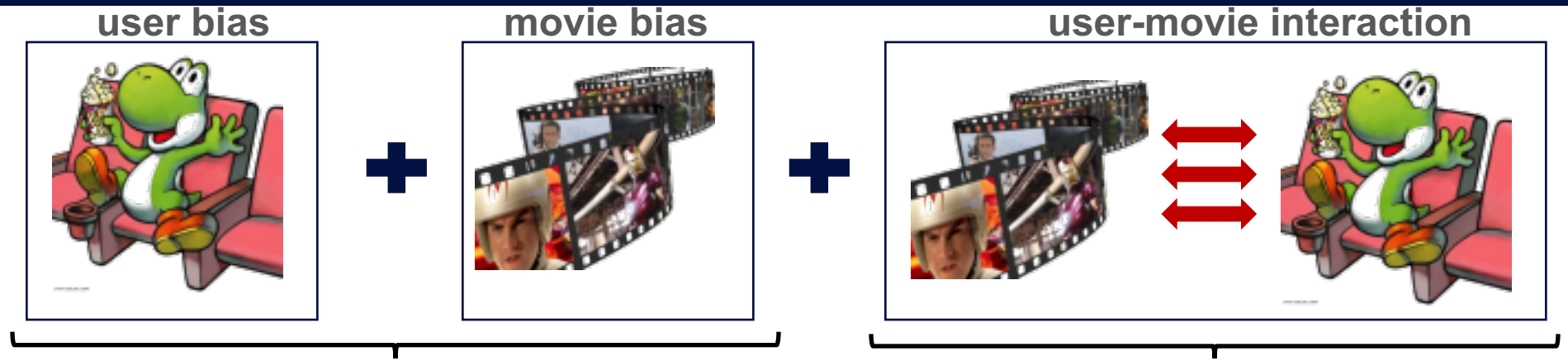
The Effect of Regularization



$$\min_{P,Q} \sum_{training} (r_{xi} - q_i p_x)^2 + \sum_x \|p_x\|^2 + \sum_i \|q_i\|^2$$

$\min_{factors}$ "error" + λ "length"

Modeling Biases and Interactions



Baseline predictor

- Separates users and movies
- Benefits from insights into user's behavior
- Among the main practical contributions of the competition

User-Movie interaction

- Characterizes the matching between users and movies
- Attracts most research in the field
- Benefits from algorithmic and mathematical innovations

- μ = overall mean rating
- b_x = bias of user x
- b_i = bias of movie i

Putting It All Together

$$r_{xi} = \mu + b_x + b_i + q_i \cdot p_x$$

- μ: Overall mean rating
- b_x: Bias for user ***x***
- b_i: Bias for movie ***i***
- $q_i \cdot p_x$: User-Movie interaction

- **Example:**
  - Mean rating: $\mu = 3.7$
  - You are a critical reviewer: your ratings are 1 star lower than the mean: $b_x = -1$
  - Star Wars gets a mean rating of *0.5* higher than average movie: $b_i = +0.5$
  - Predicted rating for you on Star Wars:
    $= 3.7 - 1 + 0.5 = 3.2$

Fitting the New Model

- **Solve:**

$$\min_{Q,P} \sum_{(x,i)\in R} \left(r_{xi} - (\mu + b_x + b_i + q_i\, p_x)\right)^2$$

goodness of fit

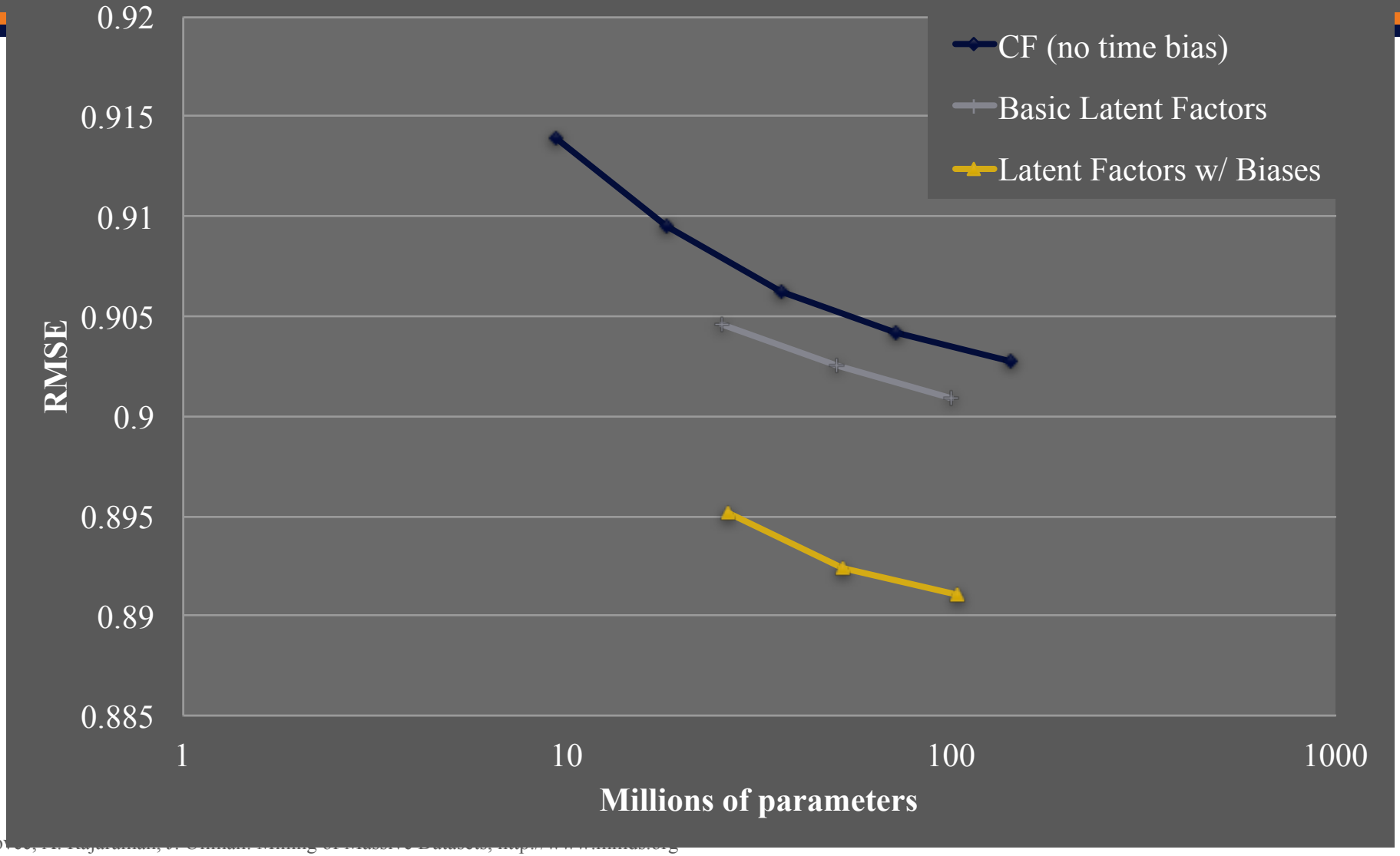
$$+\left(\lambda_1 \sum_i \|q_i\|^2 + \lambda_2 \sum_x \|p_x\|^2 + \lambda_3 \sum_x \|b_x\|^2 + \lambda_4 \sum_i \|b_i\|^2\right)$$

regularization

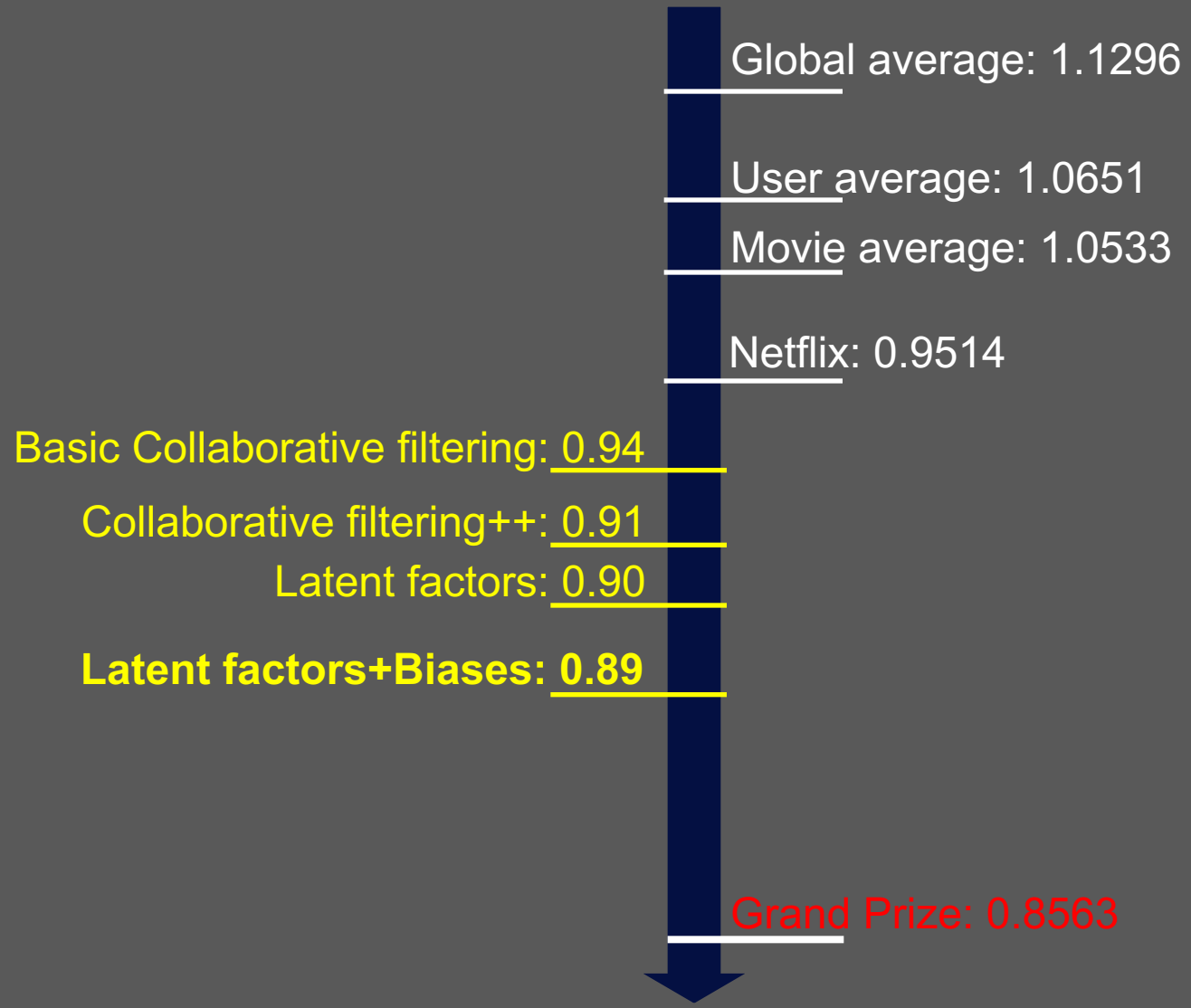
λ is selected via grid-search on a validation set

- **Stochastic gradient decent to find parameters**
  - **Note:** Both biases b_x, b_i as well as interactions q_i, p_x are treated as parameters (we estimate them)

Performance of Various Methods



Performance of Various Methods



References

- Yehuda Koren , The BellKor Solution to the Netflix Grand Prize, 2009.